



Reinforcement Learning Methods for Computation Offloading: A Systematic Review

ZEINAB ZABIHI, AMIR MASOUD EFTEKHARI MOGHADAM, and
MOHAMMAD HOSSEIN REZVANI, Department of Computer and Information Technology
Engineering, Qazvin Branch, Islamic Azad University, Qazvin, Iran

Today, cloud computation offloading may not be an appropriate solution for delay-sensitive applications due to the long distance between end-devices and remote datacenters. In addition, offloading to a remote cloud can consume bandwidth and dramatically increase costs. However, end-devices such as sensors, cameras, and smartphones have limited computing and storage capacity. Processing tasks on such battery-powered and energy-constrained devices becomes even more complex. To address these challenges, a new paradigm called Edge Computing (EC) emerged nearly a decade ago to bring computing resources closer to end-devices. Here, edge servers located between the end-device and the remote cloud perform user tasks. Recently, several new computing paradigms such as Mobile Edge Computing (MEC) and Fog Computing (FC) have emerged to complement Cloud Computing (CC) and EC. Although these paradigms are heterogeneous, they can further reduce energy consumption and task response time, especially for delay-sensitive applications. Computation offloading is a multi-objective, NP-hard optimization problem. A significant part of previous research in this field is devoted to Machine Learning (ML) methods. One of the essential types of ML is Reinforcement Learning (RL), in which an agent learns how to make the best decision using the experiences gained from the environment. This article provides a systematic review of the widely used RL approaches in computation offloading. It covers research in complementary paradigms such as mobile cloud computing, edge computing, fog computing, and the Internet of Things. We explain the reasons for using various RL methods in computation offloading from a technical point of view. This analysis includes both binary offloading and partial offloading techniques. For each method, the essential elements of RL and the characteristics of the environment are discussed regarding the most important criteria. Research challenges and Future trends are also mentioned.

CCS Concepts: • **Networks** → **Network services**; **Cloud computing**; **Network management**;

Additional Key Words and Phrases: Fog Computing, Mobile Edge Computing, Mobile Cloud Computing

ACM Reference format:

Zeinab Zabihi, Amir Masoud Eftekhari Moghadam, and Mohammad Hossein Rezvani. 2023. Reinforcement Learning Methods for Computation Offloading: A Systematic Review. *ACM Comput. Surv.* 56, 1, Article 17 (August 2023), 41 pages.

<https://doi.org/10.1145/3603703>

Authors' address: Z. Zabihi, A. M. Eftekhari Moghadam, and M. H. Rezvani (corresponding author), Department of Computer and Information Technology Engineering, Qazvin Branch, Islamic Azad University, Qazvin, Iran; emails: {zabihi, eftekhari, rezvani}@qiau.ac.ir.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.

0360-0300/2023/08-ART17 \$15.00

<https://doi.org/10.1145/3603703>

1 INTRODUCTION

With the development of **artificial intelligence (AI)**, the **Internet of Things (IoT)**, and 6G communications, many **latency-sensitive** and **computation-intensive** applications are gradually expanding [1, 2]. Examples of these applications include **face recognition**, **Natural Language Processing (NLP)**, **interactive games**, **Virtual Reality (VR)**, **Augmented Reality (AR)**, and **electronic health** [3–5]. These processes require **high energy consumption** and should be used to process **computational tasks with low latency**. The **main concern** with IoT devices (such as sensors, and cameras) or **mobile devices** is the **lack of resources** such as **computing power**, **memory**, and **battery life** [6–8]. This poses a challenge for such resource-demanding applications and is a severe **concern for Quality of Service (QoS) and Quality of Experience (QoE)** [6, 9].

Over the past two decades, **Cloud Computing (CC)** has provided remote computing and storage resources to end-user devices. In CC, tasks are offloaded to remote cloud datacenters to reduce the use of local processing resources on the user's device. This operation, known as computation offloading, reduces local power consumption in the end-device and thus extends battery life [3, 10]. However, CC also has drawbacks. Due to the **distance of cloud datacenters from user end-devices**, offloading may not be a suitable solution for **delay-sensitive applications** [4, 11]. Also, **task offloading may increase bandwidth consumption and costs**. To address these challenges, a new paradigm called **Edge Computing (EC)** emerged nearly a decade ago to bring computing resources closer to end-devices [246, 247]. Here, **edge servers located between the end-device and the remote cloud** perform the user's tasks. The edge layer mainly consists of **terminals such as smartphones, wireless access points (AP), routers, base stations (BS), edge servers, and so on**. They are also heterogeneous in the sense that **they have different hardware specifications, communication protocols, and architectures**. Because the edge servers are **very close to the user**, EC can **further reduce task response time**. EC can also **reduce network congestion and provide greater privacy**. Recently, several new computing paradigms have emerged to complement CC and EC. Some of the most important paradigms are **Mobile Cloud Computing (MCC)**, **Mobile Edge Computing (MEC)**, and **Fog Computing (FC)**. The **main motivation for the emergence of these paradigms is to process tasks close to the end devices** to reduce the response time as much as possible. They can also lead to **improved user experience and higher performance** [240]. We will discuss these complementary paradigms in more detail in Section 2.1.

Computation offloading is a **multi-objective, NP-hard optimization problem** [11, 13, 14], meaning that **it cannot be solved using classical optimization methods in a polynomial time**. Our literature review has shown that the research conducted in this area is very **diverse in system architecture, modeling, computational strategies, and problem formulation**. In a broad sense, they can be classified into three categories: (1) **Mathematical optimization algorithms such as Mixed Integer Programming (MIP)**, heuristics, metaheuristics, and game theory, (2) **AI-based optimization algorithms such as Machine Learning (ML)**, and (3) **Control algorithms such as Lyapunov optimization approaches or a combination of them**. Among them, the share of research conducted with ML methods has been significantly higher than others. ML has been used as an important enabling technology in many areas, including computer vision, medical diagnosis, search engines, and speech recognition [244]. It makes predictions by learning patterns and behaviors from historical data. One of the essential types of ML is **Reinforcement Learning (RL)**, in which an agent learns how to make the best decision based on performance metrics using the experiences gained from the environment. Here, the agent is a piece of code that performs the offloading optimization operation. It performs actions in an environment to **maximize a cumulative reward or minimize an expected loss**. Recently, **Deep RL (DRL)** has attracted the attention of many researchers. It combines the advantages of RL and deep learning. In this survey, we focus on RL and DRL techniques to solve the computation offloading problem.

Motivation. A part of the offloading optimization research has been conducted with heuristic and metaheuristic methods. However, these methods have their shortcomings. The biggest problem with metaheuristic methods is getting stuck in local optima. To obtain a globally optimal solution, they must make a trade-off between exploration and exploitation. In addition, metaheuristic algorithms have many hyperparameters, which are often difficult to adjust. These are parameters that are manually tuned by the researcher. For example, in the genetic algorithm, the crossover rate and the mutation rate should be carefully adjusted. As another example, in the gravitational search method [252], the convergence rate, represented by α , must be carefully tuned. In contrast, RL methods are self-learn and self-adaptive [15]. They have fewer parameters and better global search capability. As a result, they can be used to model more complex high-dimensional scenarios in the real world. For simplicity, most previous studies have considered the environment as static. This unrealistic assumption cannot capture the requirements of the real world. The most important sources of dynamicity are wireless channel status, user mobility, the variable arrival rate of tasks, and time-varying features of servers [16]. *→ why we call them dynamic?*

The variety of tasks and the high dynamics of the computing environment make the offloading problem more complex and difficult. Moreover, the agent does not know in advance which offloading strategy is better in an uncertain environment. Furthermore, for delay-sensitive applications, fast offloading decisions are essential [232]. In such a dynamic and complex system, traditional optimization methods may not be able to converge to an optimal solution in a reasonable time. Among the different machine learning techniques, RL and DRL are suitable options for offloading decision-making in complex dynamic environments with unknown and uncertain information. In general, RL techniques do not require complete knowledge of the system. They can learn the optimal offloading policy by directly interacting with the environment without any prior knowledge of the system dynamics. Most of the RL methods are based on the **Markov Decision Process (MDP)** [17]. It is an effective mathematical tool for modeling the agents' interactions in a dynamic environment to achieve a long-term goal. Unlike the greedy approaches used for static environments, here in RL, decisions can be made in a long-term manner in a dynamic environment [16].

Despite many advantages, RL-based offloading techniques suffer from large state/action spaces. Here, state means the environmental conditions in which the RL agent is located. Also, action means the decision that the agent makes in response to the environment. We will explain these terms in detail in Section 2.3. To reduce the dimensions of the state/action space, in recent years, DRL methods have attracted a lot of attention from researchers. It uses a **Deep Neural Network (DNN)** as the estimator to increase the scalability of RL to adapt to the edge/fog/cloud offloading environment. In this context, the estimation is a prediction of a certain quantity based on previous data. DRL can learn the optimal decision policy in real-time. Although the trial-and-error process of DRL is time-consuming, after the system converges, the optimal offloading decision is found. Optimized offloading can lead to very significant savings in resource consumption [18]. In addition, DRL is based on experience and does not make any assumptions about the environmental model. It can improve its policy during the offloading operation by accumulating new experiences. In this way, it quickly adapts to environmental changes to meet the needs of latency-sensitive applications.

The above shortcomings prompted us to prepare a comprehensive study on RL-based computation offloading. So far, several surveys have investigated the problem of task offloading optimization in different computing paradigms and from different perspectives. However, few of these studies have specifically addressed RL techniques. We categorize and review previous research based on RL in different computational paradigms such as EC, FC, and MCC from various perspectives. For each RL technique, after introducing its features and applications, we explain its

strengths and weaknesses. Then, we discuss the characteristics of offloading models in detail. We hope this survey can provide useful insight to researchers.

Comparison with Existing Surveys. In this section, we discuss recent review papers on computation offloading. Our literature review shows that different authors have studied computation offloading in different paradigms and with different approaches. Some studies have generally studied computation offloading methods [14, 19–23]. Previous studies have presented an overview of computation offloading in the MEC paradigm [24, 25]. In some studies, a closer area has been explored. For example, some have concentrated on scheduling [26, 27]. Others have examined the VM placement [28]. Another critical issue is load balancing [22]. As mentioned earlier, vehicular traffic processing and transportation systems are other application areas. The authors in Reference [29] examined the issue of offloading in the vehicular network and categorized the previous research. Here, some research has examined the issue of optimal mobility management in more detail [20]. Some studies discuss only the channel model [30]. In some research, code, and data offloading have been investigated [23]. Some studies investigate the problem from the viewpoint of optimization and decision theory. For example, Reference [24] presents a classification in four areas: classical game theory, auction theory, evolutionary games, and hybrid games. Also, the authors in Reference [31] examine various aspects of computation offloading and resource allocation. Offloading methods based on mathematical optimization, artificial intelligence, and control theory have been investigated in Reference [21]. In another study [32], stochastic-based strategies were investigated.

None of the works mentioned above have analyzed the offloading mechanisms in detail from the perspective of RL/DRL. Some of them have exclusively addressed RL/DRL algorithms in part of their analysis. For example, the authors in Reference [33] deal with the application of DRL in network and communication. Also, in Reference [34] the application of DRL in autonomous IoT is investigated. In the above studies, the main focus of the authors was not on the computation offloading problem. In Reference [25], the authors present a systematic review of machine learning approaches and divide offloading methods into three categories: supervised learning, unsupervised learning, and RL. They have only reviewed the articles that were studied in the MEC environment. In Reference [35], a comprehensive survey of the application of multi-agent RL in the future Internet is presented. Again, the main focus here is not on the issue of offloading. The authors in Reference [36] discuss RL-based resource management exclusively in the fog computing environment. Also, in Reference [18] the application of RL is presented only in vehicular task offloading in edge and vehicular cloudlet.

Unlike the above research, in this survey, we conduct a comprehensive systematic review regarding the applications of RL/DRL techniques in computation offloading for FC, MEC, and MCC paradigms. This review is based on the classification of previous RL/DRL studies. Table 1 shows a comprehensive comparison of our survey with the most important previous works.

Our Contributions. Our main contributions are summarized as follows:

- We conduct an extensive systematic review of RL-based task offloading in different computing paradigms such as MEC, FC, and MCC.
- In this survey, a complete background on each RL algorithm is provided. Then, the application of each technique in task offloading schemes in different computing paradigms is described. In addition, detailed tables are provided based on the research questions for each technique. In these tables, offloading schemes are studied from different perspectives such as offloading type, performance metrics, algorithm type, system components, evaluation tools, computing environment, and characteristics.

Table 1. A Comprehensive Comparison of Our Survey with the Most Important Previous Works

Ref.	Year	Cloud	MEC	Edge	Fog	Vehicular Network	IoT	Focus on offloading	Focus on RL/DRL	RL component (State, Action, Reward)	Paper Selection	Future Directions
[24]	2020		✓					✓			✓	✓
[14]	2021		✓					✓				✓
[19]	2020						✓	✓			✓	✓
[26]	2021	✓						✓			✓	
[27]	2019	✓						✓				
[28]	2021				✓		✓	✓				
[22]	2019	✓		✓				✓				
[29]	2020					✓		✓				✓
[20]	2021		✓					✓				✓
[30]	2020			✓				✓				✓
[23]	2020							✓			✓	
[31]	2020			✓				✓				
[21]	2021	✓		✓				✓				✓
[32]	2020	✓		✓				✓			✓	
[33]	2019								✓	✓		✓
[34]	2020						✓		✓	✓		✓
[25]	2020		✓					✓			✓	✓
[35]	2022								✓			✓
[36]	2022				✓				✓			✓
[18]	2022					✓		✓	✓	✓		✓
Our article	2023	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

- We identify challenges and open issues for RL-based offloading. Also, the future research trends for the application of AI/ML/RL in computing paradigms for MEC, FC, MCC, serverless computing, and quantum computing are addressed.

Article Organization. The organization of this article is shown in Figure 1. Section 2 presents a review of computing paradigms, computing offloading, and fundamentals of RL. Section 3 describes the research methodology and its essential questions; Section 4 elaborates on the critical RL techniques used for computation offloading; Sections 5 and 6 are devoted to discussing the results and future directions and research challenges; finally, Section 7 concludes the survey. Also, Table A.1 in Appendix A.1 summarizes the acronyms.

2 BACKGROUND

This section first provides an overview of computing paradigms, especially computation offloading. Then, it introduces a taxonomy for RL on offloading.

2.1 Review of Computing Paradigms

Cloud computing is the most well-known computing paradigm that has been around for almost two decades. As defined by the **National Institute of Standards and Technology (NIST)**, CC is a paradigm for on-demand access to configurable computing resources such as servers, storage, applications, and services [37]. In general, cloud datacenters are located far from the end-devices. Therefore, CC may not be a suitable solution for some delay-sensitive applications [21, 31, 243]. With the expansion of **Mobile Computing (MC)** in the past decade, a paradigm called MCC has

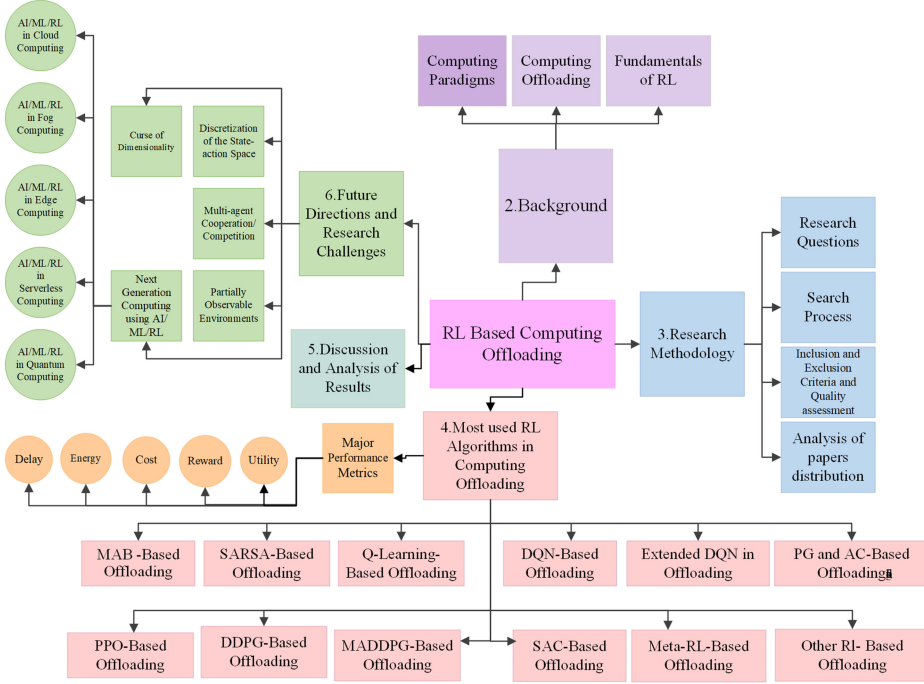


Fig. 1. The organization of this article.

spread [38, 39]. It provides computing, storage, services, and applications over the Internet at a lower cost and greater flexibility [21]. MCC consists of mobile devices, a wireless access network, the internet, and a cloud infrastructure. The purpose of the wireless access network is to communicate between mobile terminals and the Internet through BSs or APs. Tasks are delivered to the remote cloud after being transferred to the service provider's server over the Internet. In recent years, with the proliferation of latency-sensitive IoT applications, MCC alone is unable to meet real-world requirements. The main reason is that cloud resources are centralized and far from users and are not able to respond quickly. Processing a task in the MCC also requires a significant amount of energy [21, 245].

For the reasons explained above, the use of complementary paradigms for MCC such as EC is of key importance. EC provides the possibility of performing computations at the edge of the network close to the source of data [30]. For example, edge devices collect data from IoT sensors and process it locally. If the local processing capacity is not enough, then the task is offloaded to the remote cloud. Local processing instead of offloading to the cloud may provide more data security in addition to improving user experience and reducing latency [31]. Inefficiencies caused by the geographical distance of cloud servers in MCC led to the emergence of a complementary paradigm called MEC, also known as multi-access edge computing [40, 41]. It was introduced by the European Telecommunications Standards Institute (ETSI) to standardize APIs between the mobile edge platform and open services such as AR, VR, and Mixed Reality (MR). The core idea in MEC is to offload intensive and energy-consuming tasks to BS servers in the edge neighborhood. Minimizing the average execution time and energy consumption are the two basic goals of MEC. Typically, mobile devices use cellular communications to connect to the MEC server. Due to the use of adjacent servers, the delay caused by the remote cloud is decreased [42]. MEC can also be used to offload tasks in vehicular networks and the IoT as a realization of next-generation

networks such as 5G and 6G [41]. In MEC, a tradeoff must be established between the cost of local execution and offloading to the cloud.

In addition to MEC and MCC, another complementary paradigm called FC was introduced by Cisco in 2012 to resolve the problems of IoT applications [12]. The OpenFog Consortium has identified FC as a horizontal system-level architecture that distributes storage, computing, networking, and control functions closer to the user device [43]. The key idea in FC is to process part of tasks and services in fog devices (controllers, smart gateways, switches, routers) instead of offloading them to the remote cloud [21, 28]. FC significantly reduces latency and energy consumption. However, due to the heterogeneity of fog nodes, it may still be inefficient in providing QoS for computation-intensive tasks. To overcome this problem, each fog node can offload the task to neighboring fog nodes or the remote cloud. Here, choosing the best node for offloading is of critical importance [10].

One of the most important applications of computing paradigms is smart city traffic control [42]. **Vehicular Edge Computing (VEC)** integrates MEC with vehicular networks. It brings computing resources in the cloud closer to end devices (here vehicles). VEC plays an essential role in providing edge services with lower latency and higher bandwidth for vehicles. Each vehicle has its computing and storage resources to complete tasks. Here, **roadside units (RSU)** located near the vehicle act as an edge server for timely data collection, processing, and storage. When a vehicle does not have enough computing resources available, it may offload latency-sensitive tasks to edge servers. In this way, the delay and energy consumption of the vehicle is reduced [44, 45]. The main computation offloading modes in VEC include **vehicle-to-infrastructure (V2I)**, **vehicle-to-vehicle (V2V)**, **vehicle-to-vehicle-to-infrastructure (V2V-I)**, and **vehicle-to-vehicle-to-vehicle (V2V-V)** [44]. However, VEC suffers from the overload of edge servers during peak hours and the high cost of deploying RSU servers. To resolve this problem, **Vehicular Fog Computing (VFC)** has been proposed. In VFC, idle computing resources of moving/parked vehicles may be exploited for delay-sensitive tasks [45].

All of the above computation paradigms are capable of offloading delay-sensitive computations to the edge of the network [21, 240]. The common features of all these paradigms are to reduce hardware costs, reduce latency, and improve the quality of user experience [240]. The capacity of FC and MEC servers is typically much smaller than that of CC. In general, edge resources are heterogeneous and less accessible. They include resources such as routers, wireless gateways, and street cabinets [21, 240]. However, there are differences between these paradigms. According to the OpenFog Consortium, FC provides computing, networking, storage, control, and acceleration from a remote cloud to user devices. However, EC tends to operate only at the edge devices [248]. Another difference between MEC and FC is that offloading is managed by network operators. This is because in MEC the servers are located within the **Radio Access Networks (RANs)** of cellular systems [251]. Another difference is in the applications placed on these networks. FC was introduced to facilitate some areas of IoT and vertical markets such as energy, industry, transportation, agriculture, and healthcare. In contrast, MCC is more concerned with providing remote computing resources for mobile applications. However, MEC has acceptable flexibility for video analytics, location-based services, and augmented reality applications. Practically, MEC can be applied to any type of access network. Another factor in comparing the above paradigms is the delay. If the goal is to reduce delay alone, regardless of other metrics, then MEC is probably the most desirable option.

Figure 2 shows a general architecture for device-fog-cloud and device-edge-cloud. We have presented these two architectures in a three-layer format. The end-device layer includes smart devices (such as smartphones and tablets), IoT devices, and smart cars. These devices generally have constraints on energy consumption and battery life. They are also heterogeneous in the sense that they have different hardware specifications, communication protocols, and architectures. Devices

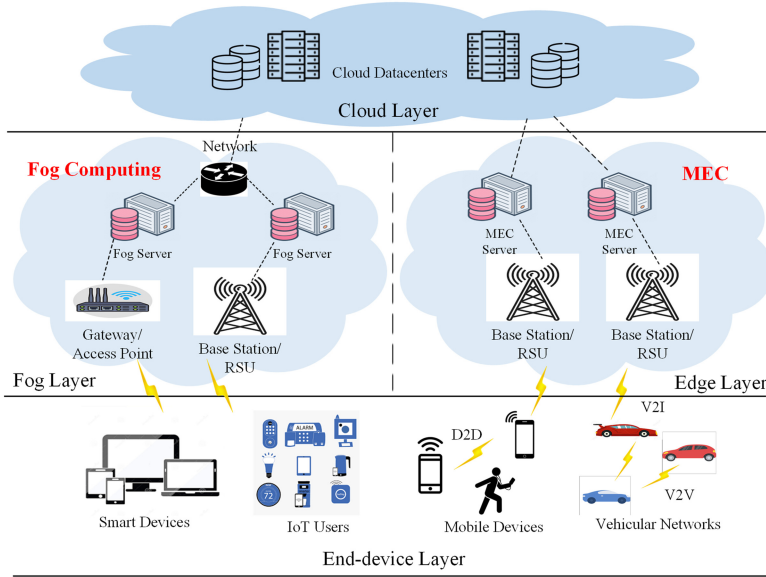


Fig. 2. The three-tier architecture of the paradigms examined in this survey.

placed in this layer are responsible for collecting environmental data and then sending it to the upper layer for processing and/or storage. A user-level node can process the request locally or send it to an edge/fog/cloud node. The fog layer consists of heterogeneous fog servers with relatively limited processing capabilities, including APs, switches, routers, gateways, BSs, and RSUs. FC is a complementary paradigm to CC that can reduce energy consumption and delay [12]. As shown in Figure 2, the fog layer is located between the remote cloud and the end-devices. It processes requests from users. If the resources of the fog server are insufficient, then it will offload the request to neighboring fog servers or the remote cloud. Typically, servers are distributed to reduce service latency [240]. The cloud layer is the final place to offload tasks. A public cloud may have dozens of datacenters, each with hundreds of servers around the world. Thus, there is no shortage of computing resources at this layer. After processing the task, the cloud server sends the response back to the IoT node.

Now, we proceed to explain the MEC paradigm in Figure 2. Here, servers are located in macro or micro BSs. By processing tasks at the edge of the network, MEC reduces latency and improves user experience. This reduces the load on the core network. The MEC paradigm consists of three layers. The smart device layer consists of heterogeneous mobile devices. These devices can exchange information with other nearby smart devices and APs located at the edge. The smart device layer is the first place where decisions are made about offloading to edge servers. At the edge layer, there are APs, BS/RSUs, and edge servers with relatively small datacenters. These servers are usually accessed by APs through powerful communication links with high data rates. The APs themselves are usually connected through optical fiber [25].

In the next section, we review studies with RL techniques in different paradigms.

2.2 Computation Offloading

Due to limited computing and storage resources, end-user devices tend to offload their tasks to remote fog/cloud servers. This is especially critical for latency-sensitive and resource-intensive tasks. In addition to improving the QoS, computation offloading can bring other benefits such as reduced cost, energy consumption, and response time. For some delay-tolerant applications,

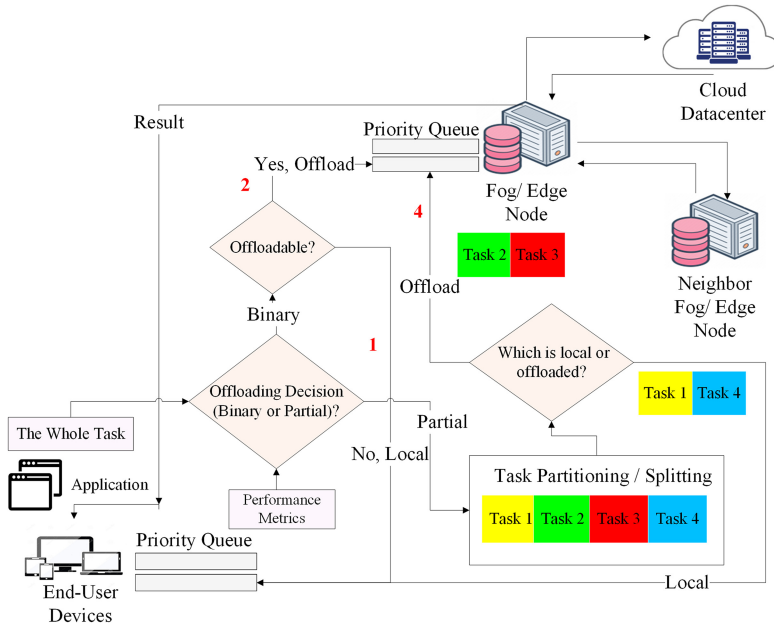


Fig. 3. Processing a task in each of the binary and partial offloading methods.

although response time is not a major concern, energy consumption is still important. For this reason, generally, offloading operations can be classified into two categories: “*delay-sensitive offloading*” and “*delay-tolerant offloading*.”

From the task point of view, the offloading decision can be divided into two broad categories: *Binary offloading (full offloading)* and *partial offloading* [14, 24, 25, 30, 32, 41, 46, 47]. In binary offloading, the end device decides to process the entire task locally on the end device or decides to offload the entire task to the fog/cloud server. In partial offloading, a part of a whole task is processed locally and the other part is offloaded. Here, a series of additional restrictions are imposed. If the selected server is overloaded or the channel conditions are not suitable, then it may take more time to offload. Also, during partial offloading, attention should be paid to the *interdependency of tasks*. Usually, task interdependency is modeled with a *Directed Acyclic Graph (DAG)* [3, 15, 48, 49] or with a *task call graph* [7].

Generally, a *tradeoff* should be *established between the cost of local execution and the cost of offloading*. When the transmission channel is in bad condition, the delay and energy consumption increase. In such a situation, offloading is not profitable. For this reason, the end device may prefer to process the entire task locally. Figure 3 shows the task processing steps in each of the partial and binary offloading methods. Here, suppose the end device is a policeman with a laptop [13]. He wants to find a criminal from a large video file. Here, the partial offloading of video to the remote cloud can lead to a significant reduction in processing time. The video file can be split into several parts, each of which can be offloaded to different servers. At first, the end device decides on partial or binary offloading by considering performance criteria. Scenario 1 in Figure 3 shows a case where the task cannot be offloaded. In this case, the task enters the priority queue for execution. Scenario 2 shows a case where the task can be fully offloaded to the remote server. In this case, the entire task is offloaded and executed *on Virtual Machines (VMs) or containers on the remote server*. Scenarios 3 and 4 show a situation where the final device has decided on partial offloading. In this case, task partitioning is done and the original task is split into several sub-tasks.

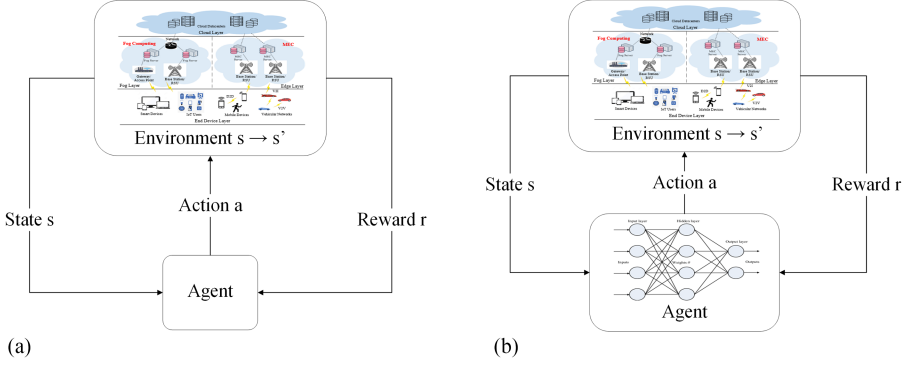


Fig. 4. Agent's interaction with the environment in (a) RL and (b) DRL.

The end device must decide which parts of the task can be offloaded and which parts must be executed locally. In scenario 3, some sub-tasks are moved to the end device's priority queue for local execution. Also, according to scenario 4, some sub-tasks are offloaded to the appropriate server for execution. Finally, the remote server returns the computation results to the end device. In all scenarios, if a fog server cannot process a task, then it offloads the task to neighboring fog servers or the remote cloud.

2.3 Fundamentals of RL

RL [17, 50] learns using the agent's experiences gained from the environment according to the predefined goals. Suppose the agent is currently in a state s . It performs an action a with continuous interaction. After the environment responds to the agent's action, it enters a new state s' and receives a reward r . The agent uses RL to find an optimal policy to maximize the expected total reward. The interaction between the environment and the agent from the initial state to the final state is called an episode or a trajectory. In other words, the agent seeks a policy that returns the highest total rewards for episodes [17, 51]. The reason for using discounted rewards is to force the agent to avoid greedy behavior and, conversely, to encourage it to engage in foresighted behavior. In other words, the rewards the agent receives in the following steps are less than the rewards it receives now [52]. RL is a trial-and-error learning method in a dynamic environment. The agent modifies its adopted strategies through environmental feedback to maximize rewards [50, 53, 54].

All previous research in computation offloading has used MDP [55]. Based on Markov's property, the future of a process depends only on observations and the current situation, and there is no need to view the complete history [56, 57]. An MDP is a stochastic control process described by a tuple (S, A, τ, R, γ) . Here, S and A denote the state space and action space, respectively. Also, τ represents the transition function $\tau(s'|s, a)$, which maps a state-action pair at a time t to the state set at time $t + 1$ or t' . In MDP, the immediate reward function is denoted by $r(s, a, s')$. Also, $\gamma \in [0, 1]$ is the discount factor. Bellman's equation states that when an agent performs an action in a state s , it enters the following state s' and receives a reward r (Figure 4(a)). In RL, a policy $\pi : S \rightarrow p(A|S)$, is a mapping of states S to a probability distribution based on selected actions A . The value of a policy π based on Bellman's equation is expressed as follows:

$$v^\pi(s) = r(s, a, s') + \gamma \cdot v^\pi(s'). \quad (1)$$

Let $Q(s, a)$ denote the value of a state-action pair. Now, the Bellman equation as a function of state-action value is calculated as follows:

$$Q^\pi(s, a) = r(s, a, s') + \gamma \cdot Q^\pi(s', a'). \quad (2)$$

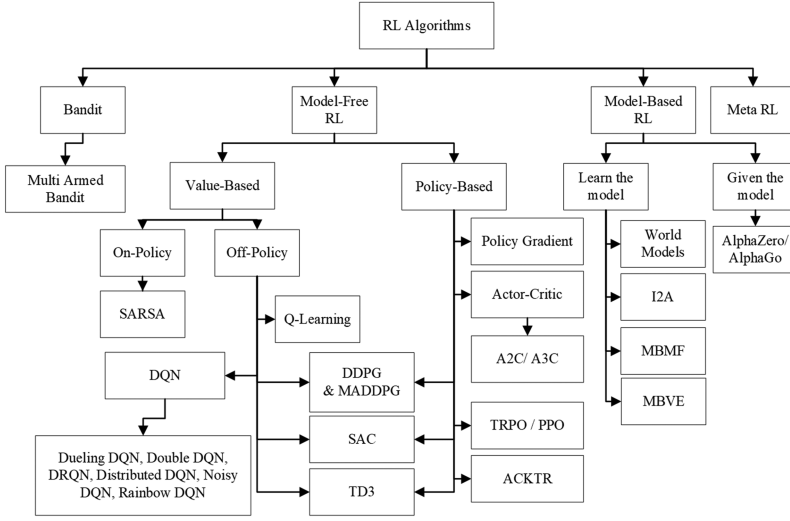


Fig. 5. The taxonomy of RL Algorithms.

If the environment is stochastic, then the agent will not necessarily go to the state s' by performing an action a . It may be in different states with different probabilities. In other words, the agent may select an action based on the probability distribution in the action space. Therefore, according to the concept of *expected value*, we write

$$v^*(s) = \max_a E_{s' \sim p}[r(s, a, s') + \gamma \cdot v^*(s')], \quad (3)$$

$$Q^*(s, a) = E_{s' \sim p}[r(s, a, s') + \gamma \cdot \max_{a'} Q^*(s', a')]. \quad (4)$$

The above assumptions apply to an MDP in which states are fully observable. **Partially observable MDPs (POMDPs)** are used when the ambient conditions are partially observable [58].

RL algorithms have a high space and computational complexity. **Deep Learning (DL)** [59] has been introduced as a new advanced technique to resolve this issue [1, 53]. It can overcome RL constraints and be used to solve complex high-dimensional problems. As shown in Figure 4(b), DRL is a method for the solution of complex, high-dimensional problems. It can be used by leveraging DNN capabilities to approximate each of the components of the value function, π policy, state transfer function τ , and reward function r [57]. DRL combines the perceptual ability of deep learning with the decision-making ability of RL. It can strongly approximate a problem with a high-dimensional state and action space leveraging a DNN. It can also accurately estimate the value function and improve the learning speed of RL algorithms [5, 53, 60]. There are several processing layers here, each of which is a nonlinear conversion. The sequence of these transformations leads to learning different levels of abstraction. For example, **Convolutional Neural Networks (CNNs)** can be used as components of RL agents, allowing them to learn directly from high-dimensional inputs [61]. Unlike supervised learning, another advantage of DRL is that it does not need to feed the entire training data set to the classifier.

Now let us briefly examine the different categories of RL algorithms. Figure 5 shows a taxonomy of RL algorithms based on References [63, 233]. RL methods are divided into two broad categories: (1) **Model-free** and (2) **Model-based**. A model should be able to reflect how the environment reacts to the agent's actions [17]. Any approach that relies on creating an MDP model is called the model-based approach, and approaches that do not use the model are called model-free. Model-based approaches require a prior understanding of the model to find the optimal policy,

which involves the availability of comprehensive data [62]. In a trial-and-error manner, the agent exploits the environmental feedback by observing a reward function R and a transition function $p(s'|s, a)$, which are usually unknown. The advantage of model-based methods is that the environmental model predicts future states and rewards. This helps the agent to plan better [63]. But in the real world, the model is usually not known in advance. In addition, the dynamics of the environment can complicate the situation. As shown in Figure 5, the AlphaGo [249] and AlphaZero [250] algorithms are model-based.

Model-free approaches are divided into **policy-based** and **value-based**. Policy-based methods update the agent policy and can manage discrete and continuous action spaces. In contrast, value-based approaches continuously adjust the value function in the training process and are more suitable for discrete action spaces. Policy-based methods may have a significant variance, and as the policy changes, a new gradient is estimated independent of previous estimates. Value-based approaches may succeed in constructing a good approximation of the value function. Still, they have no guarantee of finding the globally optimal policy and do not converge quickly.

One of the well-known RL methods is the **Actor-Critic**. It combines the advantages of both model-based and model-free and learns in parallel. The explanation of these techniques is presented in Section 4. **Dynamic Programming (DP)** is a model-based approach. Here, the optimal policy can only be found if the dynamics of the environment (transition function) are known. DP uses the Bellman equation to compute a state. According to the Bellman equation, the value of a state can be obtained by the sum of the immediate reward and the discounted value of the next state. In RL terminology, this is called bootstrapping. Thus, it is not necessary to wait until the end of the episode to calculate the value of a state. Instead, using Bellman's equation, the value of a state can be estimated based only on the value of the next state. The major limitation of DP is that it can be used when the dynamics of the environment are known in advance. In contrast, **Monte Carlo (MC)** is a model-free method. Here, sampling can be used to estimate the optimal policy without using the dynamics of the environment. The main disadvantage of the MC method is that it has to wait until the end of the episode to estimate the state or Q-value. This may be time-consuming. Another weakness of MC is that it cannot be used for continuous problems. Another model-free method is **Temporal Difference (TD)**. It combines the advantages of the DP and MC methods. On the one hand, like DP, it does not need to wait until the end of the episode to compute the state value or the Q-value. On the other hand, like MC, it is a model-free method and does not require model dynamics [17, 122].

As shown in Figure 5, another RL classification is the **on-policy** and **off-policy**. The agent in the on-policy method uses only one policy and tries to improve the same policy. Conversely, the agent uses a unique policy in the off-policy approach but attempts to improve another policy. Since in the on-policy method, the agent itself interacts with the environment, the policy through which it interacts with the environment is the same as the policy that is being improved. In contrast, in the off-policy method, the agent uses the experience of other agents to improve its policy [62, 122]. As shown in Figure 5, SARSA and Q-learning are examples of on-policy and off-policy methods, respectively. We will explain these algorithms later in Section 4.

From another perspective, RL algorithms are divided into **Single Agent RL (SARL)** and **Multi-agent RL (MARL)**. A Multi-agent system consists of a group of independent entities operating in a common environment [64]. Agents can be programmed with pre-defined behaviors. However, they often learn new behaviors online to incrementally improve system performance. MARL deals with sequential decision problems with multiple independent agents operating in a common environment. Here, the evolution of the system state and the reward received by each agent are influenced by the joint actions of all agents [65–67]. MARL can accelerate learning through parallel computing and experience sharing. It is inherently robust, because if one agent fails, then other

agents can take over some of its tasks. The main challenge of MARL is to define a good learning goal for multiple agents. Since the efficiency of the agents is correlated, none of them can perform the optimization alone [68, 69].

Most of the methods used in computation offloading are model-free. For this reason, we will not discuss model-based methods in this survey. Later, in Section 4, the formulation of model-free algorithms is presented and their advantages and disadvantages are analyzed. Then the relevant offloading methods are explained according to the research questions. The taxonomy in Figure 5 also includes the Bandit and Meta-RL methods. We will explain these methods in Sections 4.2.1 and 4.2.12, respectively.

3 RESEARCH METHODOLOGY

This study aims to investigate RL algorithms in computation offloading systematically. The methodology of this **Systematic Literature Review (SLR)** was based on the guidelines of Reference [70]. This section includes research questions, the search process, inclusion/exclusion criteria, and quality assessment.

Research Questions. To plan the review process, it is necessary first to extract **Research questions (RQ)**. These questions specify our motivation. This study faces eight key questions that underlie the search strategy for the literature review. The main research questions and motivation for raising them are as follows:

RQ1: What RL algorithms are used in computation offloading? By finding the answer to this question, we can identify how each RL algorithm is used in offloading. The answer to this question helps us identify the popularity of each RL technique in computation offloading.

RQ2: Which offloading scheme, binary or partial, is used for each RL-based approach? Is binary offloading more commonly used or partial offloading? Recall from Figure 3 that for delay-constraint parallel processing applications, partial offloading methods may be more efficient. It depends on real-world scenarios and applications.

RQ3: What performance metrics are used in RL-based offloading approaches? A literature review shows that researchers have considered various **optimization** goals and metrics. The answer to this question can determine the importance of each metric. As explained before, in general, the criteria selected in the articles have a direct relationship with the type of tasks (delay-sensitive vs. energy-sensitive). The answer to this question can also determine which categories of applications are important.

RQ4: What are the basics of MDP for any RL-based offloading approach? Has it been implemented as a single-agent or a multi-agent? As explained in the previous sections, most of the RL-based offloading modelings use the MDP. The answer to this question can determine the most commonly used states, actions, and rewards of RL techniques. Also, most dynamic computing environments work in multi-agent mode. For example, several fog servers or end devices compete or cooperate as agents. In a MARL, the reward that an agent receives after taking an action depends not only on its actions but **also** on other agents. Therefore, Single-agent RL may not be able to capture the characteristics of the real world. It does not pay attention to the decision-making policies of other agents.

RQ5: What **simulators** do researchers use to evaluate the performance of RL-based approaches? This is one of the critical questions for students and researchers. Unfortunately, experience shows that due to the lack of familiarity with the specifications and capabilities of simulators, often the wrong choices are made by young researchers in academic settings.

RQ6: What RL-based approaches are used to meet the needs of each complementary paradigm? As mentioned earlier, complementary paradigms (MCC, MEC, FC, etc.) are components of an

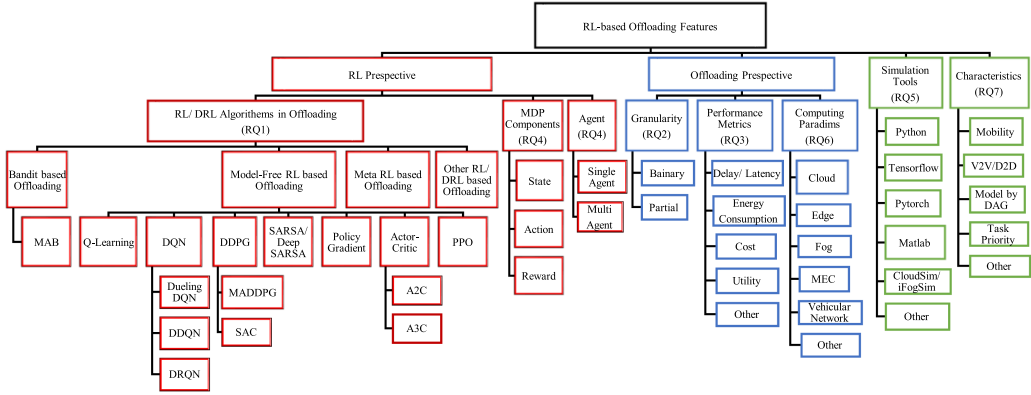


Fig. 6. The most important perspectives and research questions in this survey.

ecosystem called distributed computing. Therefore, the proposed RL solution for one paradigm should not conflict with others.

RQ7: What is the main feature of any RL-based approach? What are the main characteristics of hybrid approaches? It shows what specific ideas or characteristics have been used in these papers.

RQ8: What are the future research trends and open issues in RL-based computation offloading? Answering this question helps researchers choose a more appropriate trajectory to save resources and time.

As shown in Figure 6, we review previous studies from both the offloading and RL perspectives. The classification of Section 4 and questions RQ1 and RQ4 in Section 5 are related to the RL perspective. Questions RQ2, RQ3, and RQ6 in Section 5 are related to the offloading perspective. In addition, RQ5 and RQ7 refer to simulation tools and specific features of the approaches, respectively. Also, note that Figure 6 only examines the RL/DRL methods used in offloading.

We will respond in detail to the above research requirements in Sections 4, 5, and 6. Details of RL-based offloading methods will be provided separately in Tables A.3 to A.8 in Appendix A.1.

Search Process. The most important scientific databases in the world used in this survey are Wiley Interscience (www3.interscience.wiley.com), Springer (www.springerlink.com), ACM Digital Library (www.acm.org/dl), ScienceDirect (or Elsevier) (www.sciencedirect.com), IEEE explore (ieeexplore.ieee.org), and MDPI (www.mdpi.com). The keywords used to describe the search string consisted of terms such as “reinforcement learning,” “deep reinforcement learning,” “computation offloading,” and so on. Using keywords and combining them with Boolean “AND” and “OR” operators, the search strings were finally defined as follows:

“Reinforcement learning” or “RL”) (“Deep Reinforcement Learning” or “DRL”) (“Deep Q Network” or “DQN”) (“Q-learning”) (“DDPG” or “Deep Deterministic policy gradient” or “Deep DPG”) (“MADDPG” or “multi-agent DDPG”) (“Actor-critic” or “AC”) (“SARSA”) (“Meta RL”) (“Multi-armed bandit” or “MAB”) (“dueling network”) (“policy gradient”) (“SAC”) (“PPO” or “TRPO”) (“on-line learning”) and (“Computation Offloading” or “offloading”).

Inclusion and Exclusion Criteria and Quality assessment. Our literature review shows that after 2017, research in computation offloading has gained much momentum. For this reason, we have placed our primary focus on the beginning of 2017 until the end of 2021. The paper selection process is shown in Figure 7. Note that we have deleted papers, not in the English language or did not jointly review RL and computation offloading. Also, in the quality evaluation phase, conference papers that were not indexed or did not have a high relevance were removed. Table A.2 in

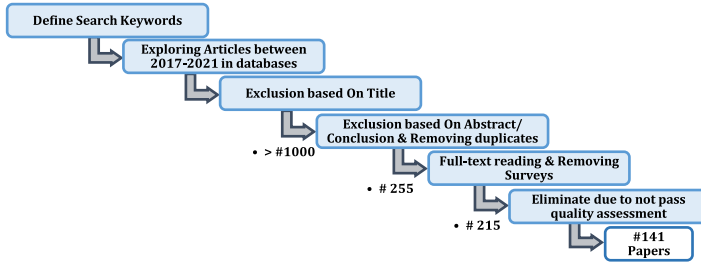


Fig. 7. The process of selecting papers in our systematic review.

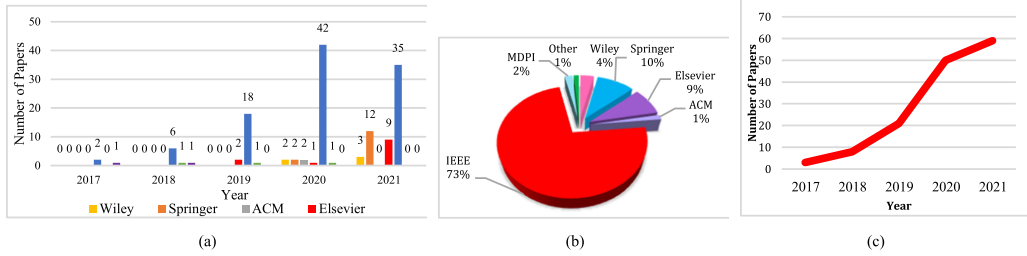


Fig. 8. Papers' statistics in this survey. (a) Distribution of papers by publishers over successive years. (b) Distribution of papers by publisher name. (c) Number of publications per year.

Appendix A.1 shows the attributes that were eventually extracted from papers. The distribution of articles published from 2017 to 2021, along with their analysis, is given in Section 5.

Distribution of Papers. Figure 8(a) shows the distribution of papers over successive years, and Figure 8(b) shows the distribution of papers by publisher name. As shown in the figure, by the end of 2021, the IEEE has the largest share compared to other publishers with 73%, followed by Springer with 10%. Figure 8(c) shows the number of publications per year. The figure shows a significant increase in RL research since 2017. Also, our literature review indicates 109 journal papers and 32 conference papers. We did a full-text reading and analyzed each of the selected articles to answer the research questions. The results of the analysis will be presented in the next sections.

4 COMPUTATION OFFLOADING MODELING USING RL

The environmental information required to design optimal offloading policies may not be available on dynamic systems. For example, IoT devices can decide based on the system's current state and the ratio of offloadable tasks. Here, the next state of the system depends only on the current state and actions and not on the history of the process. Therefore, the computation offloading problem can be modeled as an MDP, and the optimal policy can be obtained without prior network knowledge. In this section, we first review the most critical performance metrics. Then, we review the RL methods used in computation offloading.

4.1 Major Performance Metrics

Since EC/FC is an extension of CC, some of the challenges are similar. One of the key questions is: How to ensure that services are effectively delivered to end-devices under edge/fog resource constraints? In addition, offloading decisions in these paradigms can be affected by factors such as resource heterogeneity, application complexity, and wireless network heterogeneity. These factors can lead to significant degradation in QoS [242]. In general, today's IoT applications

usually have strict QoS requirements, delay sensitivity, and power constraints. However, end-devices suffer from limited computing capacity, storage, and power. Therefore, QoS-related criteria such as delay, energy consumption, and bandwidth should be considered in the design of offloading schemes [241].

Many previous studies try to jointly optimize several diverse objectives. This practically makes sense in the real world. For example, while offloading tasks can save energy on end-devices, it usually increases latency. Therefore, most previous studies have tried to find a trade-off between energy consumption and delay efficiency [11]. Our literature review shows that researchers have used different criteria to evaluate RL-based offloading techniques. Here, we mainly focus on the criteria necessary for computation-intensive, energy-hungry, and delay-sensitive applications [238, 241]. These include delay, energy consumption, cost, reward, and utility. The main purpose of offloading is to minimize task delay/ latency and save energy consumption jointly [46, 77, 78, 92, 94, 138, 145, 164, 187, 216, 228]. Unlike mobile devices, fog devices are usually connected to a power source continuously, and therefore energy conservation is not a serious concern for them. For fog devices, computational capacity is a more serious concern than energy consumption [13]. It may lead to a significant delay in the completion of tasks. In general, the criteria chosen in the papers have a direct relationship with the type of tasks (delay-sensitive vs. energy-sensitive). For example, driverless cars are one of the most well-known delay-sensitive applications. In this case, if the delay in making a decision is too long, then a serious traffic accident may occur [72].

We now explain each of the main criteria in detail. Later, in Section 5, we will present the relevant statistical information.

4.1.1 Delay/ Latency and Response Time. Minimizing delay/latency is one of the main goals in fog/Edge-cloud offloading [21, 28, 240]. In general, the offloading time $t_{offload}$ for a task is the summation of time taken to transfer the task from the end device to the edge/fog server t_{send} , the waiting time in the server queue t_{wait} , the request processing time on the edge/fog server t_{server} , the offloading time from the edge/fog server to the cloud t_{cloud} , and the response time of the server t_{resp} . Offloading a task occurs when the resources of the edge/fog server are insufficient to process it. At this time, α percent of the task is offloaded from the edge/fog server to the remote cloud [4]. Therefore, the total offloading delay is as follows:

$$t_{offload} = t_{send} + t_{wait} + t_{server} + \alpha t_{cloud} + t_{resp}. \quad (5)$$

Similarly, if an edge server's resources are insufficient to process the task, then it will be offloaded to the central cloud or other edge servers. Transmission and propagation delays in different infrastructure layers and processing and queuing delays in different devices should also be considered here.

In addition to the major sources of delay described above, some studies [21] also consider the delay caused by task optimization. In MCC, latency depends on several factors such as task size, download rate, offloading scheme, and granularity [239]. The definition of delay may vary from one study to another. For example, in Reference [234], the service delay for an IoT node is defined as the time interval between the service request and the receipt of its response. Response time is another criterion that is closely related to delay/latency. It is the time between the arrival of a task and the return of the corresponding response to the user [236]. Offloading is useful if it guarantees that the response time of a task is reduced, regardless of the effect on energy consumption [242].

In some references [5, 71–76], delay minimization is defined as a single-objective problem. For example, in Reference [74] the goal is to minimize the total long-term delay of all tasks. The authors have included the speed of the vehicle in the calculations. In Reference [71], the delay is the sum

of the round-trip transmission time and the queuing delay in the fog nodes. Since the fog nodes are usually located near the end-devices, the authors consider the queuing delay in the fog nodes to be higher than the transmission delay. In most research [46, 77–79, 164], the goal is to jointly minimize delay and energy. For example, in Reference [79], the goal is to minimize the weighted sum of delay and energy consumption. Here, delays include computation delay, data transmission delay, and queuing delay. In Reference [164], the goal is to minimize the total computational overhead in terms of the weighted sum of energy and latency. A few studies [80, 81] also jointly minimize delay, energy, and bandwidth as multi-objective problems. For example, in Reference [81], the authors assume that the total cost is related to the bandwidth allocation rate and channel utilization.

4.1.2 Energy Consumption. Because end devices typically run on batteries, the main concern is to minimize energy consumption. Energy consumed for offloading a task $E_{offload}$ is the summation of energy consumed to transfer the task from the end device to the edge/fog server E_{trans} , the energy used to process and store the task at the edge/fog server E_{proc} , the energy consumed for offloading to the central cloud (if the local resources of the edge/fog instance are insufficient) E_{cloud} and power consumption to return the response to the end device E_{ret} [4]. Therefore, $E_{offload}$ is calculated as follows:

$$E_{offload} = E_{trans} + E_{proc} + E_{cloud} + E_{ret}. \quad (6)$$

The total power consumption E_{total} for a task is different in binary and partial offloading modes. In binary offloading, the request is either executed locally or fully offloaded. In this case, the energy required for local execution E_{local} is proportional to the CPU's task size and the number of cycles. Also, the energy consumption for offloading $E_{offload}$ depends on the specifications of the remote server. A fraction of the task is offloaded in partial offloading, and the rest is executed locally [24]. Here, the total energy consumption E_{total} is the weighted sum of the local energy consumption E_{local} and the energy consumption $E_{offload}$. It is calculated as follows:

$$E_{total} = \alpha E_{offload} + (1 - \alpha) E_{local}, \quad (7)$$

where α is the percentage of the task that must be offloaded to the remote server. Devices such as cell phones have power and memory limitations. However, some applications such as social games are power-hungry, also known as energy-constrained. For these applications, it is essential to provide power to mobile devices at all times of the day. Offloading can be a suitable solution for such applications [20]. Also, in the **Internet of Vehicles (IoV)**, energy shortage is becoming a major problem. According to statistics, the transportation industry has one of the highest energy demands. This will be a major challenge in the near future, especially for **Electric Vehicles (EVs)** [145].

A review of previous research shows that minimizing energy consumption is sometimes modeled as a single-objective problem [3, 82–85, 109, 110, 124, 217]. For example, in Reference [82], the goal is to optimize long-term energy efficiency. The authors formulate the total energy consumption as the sum of the energy consumption of local execution, execution on edge servers, and remote transmission. In some studies, energy consumption optimization is modeled as a multi-objective problem [46, 77–79, 164]. In Reference [3], joint optimization of resource allocation and offloading is done by considering system resource constraints and delay threshold. Also, in Reference [109], joint optimization of vehicle-RSU communication cost and RSU-vehicle energy consumption is done.

4.1.3 Cost. The cost of a task is the total cost of transmission, the execution on the server, and the return of the response to the end device. The execution cost is itself the summation of local

execution and offloading cost. Also, the cost of processing delay and buffer delay may be considered. These costs also depend on VM placement, response time, and workload. A key technical challenge is to balance the overall computational and communication costs when making offloading decisions [16, 179]. In most computation offloading research [16, 88–96], cost refers to the sum of energy and latency costs. For example, in Reference [165], the computation cost is defined as the weighted sum of energy consumption and completion time of a task m belonging to node i . The total cost is calculated as the weighted sum of local and remote costs as follows [165]:

$$\text{Cost}_{i,m} = \alpha_{i,m}T_{i,m} + \beta_{i,m}E_{i,m}. \quad (8)$$

In Equation (8), $\alpha_{i,m}$ and $\beta_{i,m}$ are the weight of calculation completion time and the weight of energy consumption, respectively. Here, different user needs can be satisfied by adjusting the weights. For example, if $\alpha_{i,m}$ is considered larger than $\beta_{i,m}$, then the delay is more important than the energy consumption. Such a setting is suitable for latency-sensitive applications. Previous research has used many different formulations for cost. As another example, in Reference [15], a dynamic pricing mechanism based on the remaining amount of edge server resources is used. The lower the remaining amount of resources on the server, the higher the price. Here, the user tries to choose a server for offloading whose unit price is lower than the others. The main sources of cost in previous research are bandwidth cost [81, 87], server usage cost [97], monetary cost [98], and caching cost [9]. Some consider other overhead costs in addition to energy/delay costs. Some examples are energy cost plus monetary cost [99], energy cost plus bandwidth and buffer queue length [86], and service delay cost plus operating cost [100].

4.1.4 Reward. One of the metrics that researchers consider in evaluating RL algorithms is the reward [101]. It is feedback from the environment to the action taken by the agent. Based on the reward/punishment received, the agent realizes how right/wrong its action was. The reward function maps each perceived state to a value. The agent's goal is to maximize the amount of the reward. Usually, the reward function is considered equivalent to the goal of optimization. For example, if the goal is to jointly minimize delay and energy consumption, then the reward function is expressed as the weighted sum of these two parameters. Designing an appropriate reward function for an RL method is a subtle task. The reward function should reflect the real requirements of the problem and be practically implementable. Therefore, one of the metrics that can be used to compare previous studies is the subtlety of their reward function designs.

4.1.5 Utility. In economics, utility is used to model a value. It is represented as a utility function that represents a consumer's preferred order over a set of choices. Some previous studies based on pricing mechanisms have used the utility function. The utility function is the real objective that each agent tries to optimize. In Reference [117], the goal is to maximize the utility of offloading by considering the weighted sum of delay and energy. In Reference [219], a utility function is defined for each IoT device. Here, selecting an appropriate edge server depends on the value of the utility function. The authors define the utility as a function of energy consumption, computation delay, task drop loss, and data sharing gain. In Reference [222], the goal of task offloading is to maximize utility and minimize processing delay and overhead probability. In Reference [213], the utility of the VEC network has two components: communication utility and computation utility. In Reference [165], the reward function is designed based on the utility value. A higher utility value is assigned to an action that involves less delay and energy consumption.

4.1.6 Other Metrics. Because previous studies have been conducted on different paradigms, the variety of metrics is high. In addition to the metrics explained above, some studies have also used miscellaneous ones. Examples of these metrics include throughput and bandwidth. Tables A.3

through A.8 contain almost all of the metrics. Since the number of articles related to these criteria is small, we do not provide a further explanation here.

4.2 Most used RL Algorithms in Computation Offloading

This section reviews key RL-based research for computation offloading in MEC and FC and other paradigms. As previously stated, most RL techniques used in computation offloading are model-free.

4.2.1 MAB (Multi-armed Bandit)-based Computing Offloading. Unfortunately, most RL policies face the problem of *exploration* versus *exploitation*. There must be a balance between searching the problem space for profitable actions and taking the best experimental action possible. The MAB problem is one of the earliest examples of an exploration-exploitation tradeoff [102]. This algorithm has been extensively studied in statistics [103] and is also used in other fields of AI, including RL [17]. It seeks to establish a tradeoff between pursuing the best solution in the past and exploring potential new opportunities. At each step, a single resource is assigned to action in a MAB problem, and an observable gain is obtained. The goal is to maximize the total profit earned in a sequence of allocations. The agent must learn to select a single optimal arm from a set of candidate arms based on previous rewards [102, 104]. The MAB exists in both SARL and MARL variations [54]. Previous research has used MAB to deal with unfamiliar environments and learn the best task-selection strategy. Table A.3 in Appendix A.1 summarizes the MAB research in computation offloading. In Reference [71], the computation offloading in a fog network with unknown dynamics is considered, in which users can offload their tasks to neighboring fog nodes (FN). The authors consider an offloading policy with layered *exploration* and *exploitation* courses to solve the FN sequential selection problem. The offloading problem is modeled as a **Restless MAB (RMAB)** to select the right FN with a low workload. Here, FNs are considered arms, and response time is a reward. Although this algorithm can work in a completely decentralized way, it only considers the delay metric. In Reference [105], the offloading between IoT devices and FNs is modeled as a combinatorial MAB problem. The authors propose a **Learning-aided Green Offloading (LAGO)** scheme to balance *exploration* and *exploitation*. Also in Reference [106], the MAB technique is used to update the importance of tasks according to the feedback.

4.2.2 SARSA-based Computing Offloading. **SARSA** [107] stands for **State-Action-Reward-State-Action** algorithm. It is a **Temporal Difference (TD)**, model-free and on-policy method. SARSA selects an action based on its current policy and updates the current policy. In SARSA, policies that interact with the environment and updated policies are the same. It uses the ϵ -greedy or ϵ -soft approach [17] to transfer between states. *Exploration* involves choosing a random action with a ϵ -greedy policy. Also, *exploitation* means selecting the action with the highest Q-value with the probability of $1-\epsilon$. The Q-value in SARSA is as follows:

$$Q(s, a) = Q(s, a) + \alpha(r + \gamma Q(s', a') - Q(s, a)), \quad (9)$$

where α is the learning rate. It determines the effect of new information on the Q-value.

Table A.3 in Appendix A.1 summarizes the computation offloading methods using the SARSA. In Reference [8], a SARSA method based on offloading decisions is proposed to reduce energy consumption costs and time delays. One side effect of this research is the offloading location and mobility management. Also, it does not consider the dynamics of the environment. [108] presents a random computation offloading method for ultra-dense sliced RAN to maximize long-term utilization. Offloading decisions are based on queue status, energy, and channel quality. This research uses the DQN-based RL combined with Deep SARSA to deal with the large state-space dimensions. Unfortunately, the authors do not consider mobility management and its impact on states and

reward function. Also, in Reference [109], a combination of SARSA and fuzzy logic is used to resolve the problem of high dimensions due to the increased number of vehicles. It can improve long-term rewards compared to Q-learning.

4.2.3 Q-Learning-based Computing Offloading. Q-learning [51, 52] is a model-free and off-policy method. Like SARSA, Q-learning aims to estimate Q-values for an optimal policy and the best possible action. Like most off-policy methods, it uses two different approaches to behave in the environment and find the optimal approach. Unlike SARSA, which uses an ϵ -greedy policy to select both actions in the environment and find the optimal policy, the Q-learning method uses a greedy policy to choose the Q-value of the next state-action pair. Accordingly, it determines an action that results in a maximum Q-value. It, like SARSA, uses a ϵ -greedy policy to update the Q-value. The Q-value in Q-learning is as follows:

$$Q(s, a) = Q(s, a) + \alpha \left(r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right). \quad (10)$$

The only difference between the above formula and the SARSA formula is $Q(s', a')$. In Q-learning, all compounds of $Q(s', a')$ are examined, and the action with the highest Q-value is selected. For this reason, Q-learning uses the State-Action-Reward-State function to estimate the Q-value. Q-learning is one of the most widely used RL algorithms and can achieve the optimal policy for the agent without needing prior knowledge of the environment. However, it has low scalability, because if the state-action space of the problem is large, then the computational complexity increases exponentially. Also, as the number of state-action pairs increases, keeping all Q-values in a table is a big challenge.

Table A.3 in Appendix A.1 summarizes the Q-learning research in computation offloading. Some studies on edge networks and MEC paradigms have used the Q-Learning algorithm to solve the offloading problem. For example, in Reference [48], Q-learning is used for adaptive learning of optimal offloading policy and hidden task dependencies without prior knowledge. Here, the Taylor polynomial approximates the transmission delay between the end device and the edge server. Many researchers in offloading have used the delay parameter. In Reference [110], offloading and scheduling were used jointly to minimize the energy consumption of end devices. Double Q-learning with intervention is used to ensure that real-time tasks are met. The novelty of this research is to examine the policies needed to facilitate learning convergence. However, it only involves energy consumption in the reward function. In Reference [111], the offloading problem in the MEC is targeted considering the dynamics of the wireless channel. The authors propose two greedy algorithms and RL to maximize the average utilization. [112, 113] propose the Q-learning algorithm in MEC-based IoV to minimize latency and computing costs. This real-time energy-aware scheme determines the preferences of vehicle passengers based on their battery capacity. Although the authors consider V2V communication, they do not implement it in a real IOV environment. Also, [114] proposes a three-tier architecture in which end devices can offload tasks to **Unmanned Aerial Vehicles (UAVs)** or cloud servers. In another study [115], the authors use an experience replay buffer to maintain the unfinished learning temporarily experiences. They also propose a reward rewind match strategy to update the Q table. In Reference [116], optimization solutions such as pre-reward, small learning targets, and great reward sensitivity were proposed to improve Q-learning. In another study [2], the authors also considered mobility management to select the optimal nodes. Unfortunately, this method suffers from long pre-scheduling.

Q-learning has been used in other paradigms as well. In Reference [117], end devices compete to access edge servers in **Power IOT (PIOT)**. In Reference [77], an intelligent offloading scheme using rank-based Q-learning was proposed to minimize energy and latency. A smart gateway starts the RBQ-learning algorithm to balance the load between FNs. Initially, each FN's load and energy

level are monitored by a gateway, and then offloading starts. Also, [118, 119] presents a Q-learning WIFI offloading method based on user connection history to enhance QoE. Users avoid connecting to networks that have caused them bad experiences and long delays.

4.2.4 DQN-based Computing Offloading. Q-learning can find an optimal policy when the state-action spaces are small. Unfortunately, calculating values for all pairs and storing them all in the Q-table will be costly. If the agent could observe all the states and know their Q-value, then it could make the optimal decision. But this is often impossible, because the agent cannot generalize their previous experiences when faced with a new state. Another disadvantage of Q-learning is that due to the application of the MAX operator to the estimated value of the action, values that are overestimated take precedence over values that are underestimated. Such overestimations associated with noise can lead to weak policies [120]. An approximating approach such as **Neural Network (NN)** is used to overcome this problem. **Deep Q Network (DQN)** [121] is one of the most critical DRL algorithms that combines Q-learning with deep CNN. This model-free and off-policy algorithm uses multilayer NNs to generate the value-action vector $Q(s, a; \theta)$ as the output, where θ is the CNN parameter. The network is then trained in several iterations to obtain the optimal θ . In this way, the Q function and policy will be optimized.

One of the problems with DQN is that due to the high correlation between the inputs, the convergence of it decreases, resulting in overfitting. A replay buffer can store past experiences in the form of $Q(s, r, a, s')$ movements to overcome this problem. Thus, instead of using only recent experiences, the agent randomly samples from a mini-batch and uses it for training. This makes it possible to reduce the variance of network parameters and data correlation and improve convergence by repeatedly using previous experiences. It also reduces the computational cost without losing much information [62, 63, 122]. **Experience Replay (ER)** is a fixed-size buffer that can be implemented as a FIFO queue or **Prioritized ER (PER)** [123]. It prioritizes transmissions with a higher TD error, because the agent learns more from them.

Another problem with DQN is that the target values (optimal Q-value) and the predicted Q-value (online value) depend on the θ parameter. Therefore, when θ is updated, the target and predicted values change in the same direction. Also, their correlation is high, because training data is generated from similar networks [62]. To make the training more stable, a NN called the target network is created with the same main network structure and with the parameter θ' . The target network is frozen for a while, and only the predicted value is updated. Periodically, in a specified number of steps, the parameters of the main network θ are copied to the target network θ' and synchronized and updated. This allows the algorithm to converge faster [17, 62, 63]. Using these two, DQN has stabilized value functions by approximators. This feature has significantly improved, especially in the Atari game [121].

Mean-squared Error (MSE) is commonly used in DQN as the loss function. Therefore, the network can be trained by minimizing the error between the target and predicted Q-values. The loss function in DQN is defined as follows:

$$L(\theta) = E[(y - Q(s, a|\theta))^2], \quad (11)$$

in which the target value is $y = r + \gamma \max_{a'} Q(s', a'|\theta')$.

Table A.4 in Appendix A.1 summarizes the DQN research in computation offloading. In Reference [79], an advanced DQN is proposed to minimize the total weighted latency and energy. Here uses a PER mechanism to select samples for network training. Each sample is given a weight based on its participation. Also, an expert buffer mechanism is used to solve the cold start problem, in which high-quality pre-processed samples are used to accelerate training in the early stages. Similarly, **Stochastic Prioritized Replay (SPR)** is used in Reference [124] to accelerate training time

and improve convergence stability. In Reference [125], a **Device-to-Device (D2D)**-MEC network model is proposed to minimize the weighted sum of explicit and implicit costs. In Reference [126], DQN is used due to the unknown dynamics of the environment. This algorithm receives the channel's current state and the task's characteristics and learns the best strategy. DQN has also been used in Reference [46] to find a near-optimal solution. The purpose of the problem is to minimize the cost of delaying the completion of tasks, the cost of production of the cloud server, and energy consumption. A similar study [78] aimed to optimize performance latency and power consumption jointly. In Reference [15], a DRL method is proposed to solve the offloading problem with several constraints in the MEC. Such a fine-grained offloading scheme, when modeled with MDP, can reduce network latency, energy, cost, and usage. The authors analyze the results of several candidate networks to overcome the problem of the overestimation of the DQN. Although this research considers inter-dependencies between tasks, it does not address user mobility management. In Reference [127], DQN is adopted for joint optimization at the device and edge levels.

In Reference [1, 89], Q-learning and DQN, are used for the offloading problem. In Reference [89], Q-learning is used to optimize the total cost of latency and energy, but as the number of users increases, DQN is used to continue working. The authors add a pre-classification step before learning to limit the action space. In Reference [1], a MEC offloading model is proposed, which uses both algorithms to reduce time overhead and energy. In Reference [128], an end-to-end DRL method is proposed to optimize UAV trajectory control and task offloading. Here, the agent maximizes the number of completed tasks to minimize energy and latency. Also, in Reference [129] complexity of state information increases as the number of end devices increases. Due to constant environmental changes, the authors use DQN to solve the problem. Similarly, in Reference [88], a DQN-based offloading algorithm is proposed in the blockchain network. Each mobile user acts as a miner and offloads IoT tasks to the MEC server. This study aims to achieve an optimal offloading map for all miners to minimize the cost of the system. DQN enables miners to perform optimal action based on current blockchain transaction data and channel status without prior knowledge.

Some research has placed more emphasis on energy-efficiency. One study [130] divided offloading requests into delay-tolerant and non-delay-tolerant. DQN determines that these requests are processed on the edge server or the cloud server. In Reference [131], a collaborative offloading mechanism between MEC servers and the cloud is proposed considering the handover. Also, in Reference [132], the DQN-based online algorithm is used to deal with the high dimensions of the state space. In Reference [133], DQN facilitates the training process, and the reward clipping method is used to prevent model fluctuations. Also, Reference [134] proposes an optimal service offloading in the virtual MEC in which IoT devices are used as virtual edge servers. First, deep learning models predict the mobility and energy of the IoT device, then an **Integer Linear Programming (ILP)** is used for **Virtual Network Functions (VNF)** offloading. The authors improve this technique with the DQN approach.

Our literature review shows that some research on DQN has been conducted in vehicular networks. A **Software-defined Network (SDN)** combined with VEC is proposed in Reference [135]. The authors aim to achieve an optimal offloading decision for all vehicles using DQN. Although this study considers the dynamics of the environment, it does not address energy consumption. It exploits V2V communication only to facilitate the offloading of tasks to the edge server. Failure to use nearby vehicles for offloading may negatively affect rewards and actions. Contrary to Reference [135], the authors in Reference [72] consider the idle resources of mobile vehicles as a resource pool. Here, the sub-tasks are offloaded to the surrounding vehicle, and the results are returned to the requested vehicle. Here the task division problem is a Max-Min problem converted to an MDP. DQN and **Genetic Algorithm (GA)** are used to find the best offloading policy and assign tasks to vehicles. Another study [136] used a DQN-based offloading strategy based on

Simulation Annealing (SA) algorithm for **Connected and Automated Vehicle (CAV)** applications in a VEC environment to reduce the offloading failure rate. Also, in Reference [137], a mathematical offloading model for an in-vehicle communication network based on AHP-DQN is presented. In Reference [138], a DQN-based task migration method is proposed to minimize system cost by considering bandwidth constraints.

In addition to MEC, DQN has been used in other paradigms such as FC, CC, and IoT to solve the offloading problem. Table A.4 in Appendix A.1 summarizes them [5, 73, 90, 99, 219, 222–225].

4.2.5 Extended DQN in Computing Offloading. This section explains some of the extended DQN algorithms, including DDQN, Dueling Network, and DRQN.

DQN and Q-learning suffer from overestimating the Q-value, which may lead to a sub-optimal policy. Estimated errors due to environmental noise, function approximation, or nonstationary can cause positive and upward bias. The presence of the MAX operator in the Bellman equation makes the Q-value estimate too optimistic, because it uses the same values to evaluate the action. For this reason, the authors in Reference [120, 139] separate the action selection phase from the evaluation phase. Here, the action is selected based on the Q function with the parameter θ and the value of Q is calculated using the Q function with the parameter θ' . In the literature, this method is called **Double DQN (DDQN)** [120]. In DDQN, the values of all state-action pairs are calculated and updated at each step using the Q function and the θ . The argmax function is then applied to all state-action values and an action a' corresponding to the maximum value of Q is selected. Finally, for this action, the Q-value is calculated using the target network with the parameter θ' as follows:

$$y_t^{DoubleDQN} = r_{t+1} + \gamma \cdot Q \left(s_{t+1}, \operatorname{argmax}_a Q(s_{t+1}, a; \theta_t), \theta'_t \right). \quad (12)$$

DDQN performs better than DQN in terms of policy accuracy and quality and can simultaneously address the issue of overestimation and instability in target values. Generally, although Double learning methods do not increase the number of calculations per step, they increase the amount of memory usage [17].

The value of an action in a given state, $Q(s, a)$, consists of values $v(s)$ and $A(s, a)$. The value function $v(s)$ estimates the importance of being in a particular state s regardless of the action. Also, the advantage function $A(s, a)$ shows how good action a is compared to the average action in state s [140]. The agent must know what to do in some states. But in many other states, the choice of action does not affect the environment. Therefore, we do not need to estimate the Q-value for all states. A **dueling network** [141] is a non-sequential deep learning architecture that estimates state values and action benefits separately. There are two streams of fully connected layers in the Dueling network architecture after the convolution layers. The first stream is the value function $v(s)$, and the second is the advantage function $A(s, a)$. The two streams are combined through an aggregation layer to obtain the value of the output Q function for each action, as follows:

$$Q(s, a; \theta, \alpha, \beta) = v(s; \theta, \beta) + A(s, a; \theta, \alpha), \quad (13)$$

where θ is the convolution layer parameter. Also, α and β are fully connected layers. Because the output of the Dueling network is a Q function, it can be trained with many algorithms such as DDQN and SARSA. Another advantage of Dueling architecture is the acceptable performance in scenarios with large action space.

A **deep Recurrent Q Network (DRQN)** [142] is similar to a DQN but has recurrent layers. A piece of limited environmental information is available in POMDP. Not all states are observable in such an environment, providing only partial information. Keeping information about previous states in memory helps the agent better understand the nature of the environment and find

the optimal policy. Therefore, recurrent NNs can be used to understand and retain information about past states [10]. In DRQN, an LSTM layer is used to replace the first post-convolutional fully connected layer of DQN. Unlike DQN, which uses a fixed-length history, the recurrent structure integrates an arbitrarily long history to better estimate the current state. Thus, it estimates the function $Q(o_t, h_{t-1}; \theta)$ instead of $Q(s_t, a_t)$. Here, θ represents the whole network parameter. Also, h_{t-1} represents the output of the LSTM layer in the previous step, i.e., $h_t = \text{LSTM}(h_{t-1}, o_t)$.

Table A.5 in Appendix A.1 summarizes the extended DQN research in computation offloading. In Reference [80], a scheme based on DDQN is proposed, which adaptively selects the offloading mode and bandwidth rate. Similar to the author's previous work, Q-learning was first used in Reference [143], but due to the curse of dimensionality, the DDQN-based method was used later. A DRL-based offloading algorithm is proposed to minimize the latency cost and dropped tasks [144]. The authors used DDQN and Dueling DQN algorithms and the LSTM technique to estimate cost and improve learning. Another study [81] used adaptive learning for the offloading policy. The K-means algorithm was used to pre-classify tasks and reduce the dimensions of the action space.

Some previous research has been conducted on vehicular networks. In Reference [94], DDQN is used for full offloading of the task in case of real-time changes in network state due to user movement. In Reference [145], the energy-efficient task is proposed on a three-layer network in IoV. Also, in Reference [146], a method for offloading network tasks with large volumes and heavy calculations in the vehicular network has been proposed considering the traffic size and computational load. In a similar study [147], a dynamic pricing scheme was proposed to motivate the sharing of idle resources in vehicles. This distributed V2V computation offloading framework is based on DDQN and aims to maximize the average utilization. In another study [148] conducted on 5G-enabled vehicle networks, distributed DDQN was used to schedule cellular channels to reduce computational overhead between vehicle and macro-cell.

DDQN method has also been used in mobile networks. For example, Reference [149] uses a combination of DDQN and dueling networks to improve the bandwidth performance of cellular infrastructures and resolve temporal locality. In another study [150], DDQN is used to increase the performance of uplink offloading in the mobile network. The **Mobile Data Offloading Protocol (MDOP)** in these two papers uses three methods: time-wise offloading, place-wise offloading, and link-wise offloading. The optimal offloading policy is selected from these three methods depending on the tolerable latency and channel condition. Similar to Reference [149], in Reference [151], Double-Dueling DQN is combined with blockchain to prevent malicious offloading access. In Reference [152], a heuristic is used to reduce the complexity of Dueling DQN using preprocessing. DDQN method has also been used in Satellite networks [226].

4.2.6 Policy Gradient (REINFORCE) in Computing Offloading. Policy gradient (PG) [153–155] is one of the most popular algorithms in DRL and a policy-based method. In policy-based methods, the state is given as input to the network and returns the probability distribution on the action spaces. The action is selected based on a random policy of the NN output. Because PG methods do the learning directly on the parameterized policy function, they are more suitable for high-dimensional or continuous action spaces. In addition, PG methods can naturally model stochastic policies and use gradient information to guide optimization. So, they generally have better convergence properties [63]. The core idea of PG is to directly adjust the optimal policy parameter θ in the NN. Thus, the network returns the correct probability distribution in the operating space. This maximizes the objective function $J(\theta) = E_{\tau \sim \pi_{\theta}(\tau)}[r(\tau)]$ by taking steps in the gradient direction $\nabla_{\theta} J(\theta)$ [156]. The $\nabla_{\theta} J(\theta)$ is obtained as follows:

$$\nabla_{\theta} J(\theta) = E_{\tau \sim \pi_{\theta}(\tau)}[\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) r(\tau)], \quad (14)$$

whose value for N trajectories is equal to

$$\nabla_{\theta} J(\theta) = \frac{1}{N} \sum_{i=1}^N \left[\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) r(\tau) \right]. \quad (15)$$

After calculating the gradient by descent approach, the optimal θ is obtained as follows:

$$\theta = \theta + \alpha \cdot \nabla_{\theta} J(\theta). \quad (16)$$

In this way, the route generation policy is improved by updating θ at each iteration. The network returns the average policy and action variance as outputs and selects an action by sampling from a Gaussian distribution. PG methods often differ in how Q^{π} is estimated, such as the REINFORCE algorithm [154], which uses a recursive relation $r^t = \sum_{i=t}^T \gamma^{i-t} r_i$. It uses a baseline, a function dependent on the state s , to avoid high variance in estimating the gradient.

4.2.7 AC-based Computing Offloading. **Actor-Critic (AC)** methods [17, 157, 158] are model-free and on-policy based on simultaneous online estimation of two structures, *Actor* and *Critic*. The AC method benefits from both value-based and policy-based methods. The actor corresponds to an action selection policy that maps states to actions. It tries to choose the best action following the policy of the current state. The actor's role is to find the optimal policy, while the critic is a predictor of the state-value function. The critic, over time, learns the value of a state in terms of the expected cumulative rewards following the actions taken by the actor. It evaluates and guides the policy produced by the actor. In AC, at each episode, the network parameter is updated. Unlike the *REINFORCE with baseline* method, where the network parameter is updated at the end of an episode, bootstrapping is used here. The value of the state is estimated by adding the immediate reward and the discounted value of the next state; thus, the learning process is accelerated. The $\nabla_{\theta} J(\theta)$ can be calculated as follows:

$$\nabla_{\theta} J(\theta) = \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) (r + \gamma \cdot v_{\phi}(s'_t) - v_{\phi}(s_t)). \quad (17)$$

Like the actor, the critic's parameter is updated at each episode. Critic's loss is the difference between the target value and the predicted value of the state. It is obtained as follows:

$$J(\theta) = r + \gamma \cdot v_{\phi}(s'_t) - v_{\phi}(s_t). \quad (18)$$

At each episode, an action is selected based on the updated policy. The critic also improves the actor's evaluation in each episode as follows:

$$\phi = \phi - \alpha \cdot \nabla_{\theta} J(\theta). \quad (19)$$

AC algorithms are slightly slower in the learning phase but perform better than Q-Learning. AC converges more easily than value-based methods. It is more stable than the policy-based methods.

The AC algorithm has many variants, two of the most famous of which are: **Advantage Actor-Critic (A2C)** and **Asynchronous Advantage Actor-critic (A3C)** [159]. They differ from the primary AC in that their focus is on learning in parallel. They are of the MARL type, meaning that they use multiple agents to interact with their environment, and each agent contains a copy of the environment. There are two types of networks: global agent and worker agent. Several worker agents or actor-learners that follow the AC architecture gather experiences from the environment. These experiences are then aggregated and sent to the global agent. The global agent integrates learning. The worker agents calculate the loss of the actor network (policy loss) and critic network (value loss) [82, 122, 160]. In A2C, a coordinator collects all the worker's experiences. After updating the global actor, each worker syncs with the global agent. To improve, the squared TD error

criterion is used as a learning loss in the critic. After gathering a trajectory, the gradients are returned to the global agent instead of updating the local actor and critic. The coordinator aggregates the workers' gradients and updates the global models. Then, both the updated actor and critic are re-synched with all the workers. A3C is an asynchronous distributed version of A2C. In this way, each worker interacts directly with the actor and the global critic. Each time a worker completes the gradient calculation, the global critic is updated by the corresponding global agent. Therefore, the computational efficiency of A3C is generally better than that of A2C [63]. Due to parallel execution, these algorithms can explore different parts of the environment with various policies. This way, the updates are less correlated in time and therefore do not require replay memory. This, in turn, reduces training time [82, 160].

Table A.6 in Appendix A.1 summarizes the PG and AC research in computation offloading. First, we describe the research that has been conducted with pure AC. In Reference [97], the online PG-based AC algorithm controls the state and action space. In AC, a policy is updated instead of an episode. This reduces the number of samples needed for training and, in turn, speeds up the learning process. The goal is to minimize servers' total latency, energy, and usage costs. Also, in Reference [7], an AC-based DRL framework is proposed to optimize offloading decisions and local CPU frequencies. On the actor side, action generation is done by a Gaussian noise-added order-preserving method to create variety in offloading actions. This method calculates each actor's action faster and more accurately, which improves the algorithm's complexity and convergence. The authors also propose a heuristic method to expedite computations using the one-climb policy to reduce the number of performance evaluations for actions. In References [161, 162], DRL-based algorithms are used to solve the offloading problem in the MEC. In Reference [161], DC-DRL increases the diversity of experience and improves the performance of distributed DRL agents. Here, a combination of deep neuro-evolution and PG with n-step adaptive learning is used to improve convergence. The results show that exploration costs and final system costs are reduced. In Reference [162], the Lyapunov optimization and DRL are combined. It first solves the Lyapunov optimization sub-problem to separate the multi-step random **Mixed Integer Non-linear Program (MINLP)**. It then integrates model-based optimization and model-free DRL. Similar to Reference [7], a noisy order-preserving quantization method is adopted to generate offloading actions. This causes a balance in the DRL, because parameters are adjusted adaptively during the training process.

In References [163, 164], a combination of AC and DQN is used to solve the problem of large-scale computation offloading. In Reference [163] DDRLO algorithm is proposed to increase computational performance to deal with jamming and interference. It uses an actor network to select the offloading policy based on edge devices' latency, power consumption, and bandwidth rates. It then uses a critic network to update the weights of the actor network for each offloading policy to reduce exploration time. In Reference [164], a three-step ACDQN is used. Initially, the agent interacts with the environment and gathers experiences. Now, the agent learns the optimal offloading policy. Similar to Reference [164], DQN and DDPG are used in Reference [91]. The actor creates continuous actions for each server, and then the critic evaluates them and outputs the discrete actions.

In Reference [74], a MARL AC-based algorithm is proposed to minimize the delay. Actor and critic networks are centrally trained in the datacenter, and vehicles make offloading decisions distributedly. This method reduces the instability of the multi-vehicle environment. Similar to Reference [74], a MARL has been used in the UAV network [165]. Each agent creates the actor network without using other agents' policies. The agent then shares the Q-values with its neighbors to get a better cumulative reward. In Reference [166], the combination of AC with quantum theory is used to accelerate convergence. The authors adopt the method mentioned in Reference [167] for modeling the dependencies among tasks. In Reference [168], a game with multi-agent POMDP is used. The decentralized DRL algorithm determines the offloading strategy through game

history without prior information from other users. This algorithm combines a PG approach with Differential Neural Computer.

We now review research that has used A2C and A3C for computation offloading. In Reference [82], A2C is proposed for the joint optimization of offloading decisions, transmission power, and bandwidth in IIOT. To implement A2C, Digital Twin Networks (DTN) launches a global agent and several learning agents. The learning agent is located in small cell base stations (SBS) and can interact with the environment. After each learning step, agents send the accumulated updates to the Macro Base Station (MBS)-based global agent, updating θ_π and θ_v asynchronously. Multi-user Edge-assisted Video Analytics Offloading (MEVAO) is targeted in Reference [169] and is formulated as a game theory problem to achieve a stable state among users. Nash Equilibrium (NE) [170] is calculated. Since users are reluctant to share their personal information, authors suggest A2C for it. Also, in Reference [98], the offloading problem in mobile IoT is formulated as a constrained MDP.

As mentioned earlier, one of the most popular AC methods is the A3C. In Reference [83], A3C obtains the optimal solution for the offloading decision and controls the downlink transmission power to minimize the system's long-term energy consumption. In each agent, two DNNs are used to approximate the random policy function by the actor and the value function by the critic. Multiple DNNs are trained asynchronously using gradient descent. This leads to increased convergence speed and improved efficiency. Similarly, in Reference [171], a resource allocation framework for MEC-enabled blockchain is developed. In Reference [160], a Knowledge-Driven (KD) offloading decision-making framework is proposed to reduce the execution time of tasks in IoV. The KD framework creates a basic decision model for each service with a different number of tasks. The vehicle uses asynchronous online learning using the A3C algorithm. From the reward received at each time slot, the offloading decision-making framework adapts to environmental changes. In Reference [172], to prevent malicious attacks on 6G vehicles, an asynchronous A3C algorithm is used to score the reputation of edge servers.

Due to space limitations, the continuation of this section is moved to Appendix A.2. It includes PPO-based, DDPG-based, MADDPG-based, SAC-based, Meta-RL-based computation offloading, and others.

5 DISCUSSION AND ANALYSIS OF RESULTS

In this section, we answer the research questions and highlight the practical implications of the future. We answer RQ1 through RQ7, which forms the basis of our survey. Due to space limitations, the continuation of this section is transferred to Appendix A.3.

6 FUTURE DIRECTIONS AND RESEARCH CHALLENGES

Generally, in computation offloading, fundamental challenges are (1) task selection, partitioning, tasks interdependency, task prioritization, and scheduling; (2) selecting the appropriate server to offload; (3) determining offloading ratio; (4) mobility management; (5) efficient allocation of resources; (6) energy consumption; (7) security issues. In addition, some of the challenges are related explicitly to the RL techniques that fall within the scope of this survey. RQ8 identifies open issues and future research trends in RL-based computation offloading.

RQ8: What are the future research trends and open issues in RL-based computation offloading? Based on the literature review, we have identified broad areas as open issues and future directions, which are as follows:

- **Curse of Dimensionality:** Unfortunately, most computation offloading environments are complex and dynamic. The most important sources of dynamicity are time-varying channel conditions, the unexpected arrival of tasks, heterogeneity of tasks, limited bandwidth,

limited device energy, and heterogeneity of processing capacity. Such dynamism complicates the state-action space and creates a curse of dimensionality. Combining DRL techniques with DNN can reduce this complexity to some extent. Some studies [81, 89, 90] have used pre-classification to limit the space to accelerate convergence. Other solutions include state preprocessing [133, 189], states' space compression [219], quantization to generate action [7, 162], use of exploration strategies to reduce action space size [214], adaptive GA [218], and the use of noise to enhance exploration [86, 87]. One future trend is combining DRL with other ML techniques to reduce the curse of dimensionality.

- **Discretization of the State-action Space:** Most RL techniques, such as DQN, are designed for discrete action space. However, many real-world RL parameters are continuous. Examples are time-varying channel conditions, buffer status, consuming energy, and the ratio of a task to be offloaded. To overcome this problem, continuous actions must be discretized [91]. Here, a coarse discretization causes a loss of accuracy. Conversely, fine discretization increases the curse of dimensionality and computational complexity. Finding the right combination for the discretization of the state-action space is one of the future hot topics.
- **Multi-agent Cooperation/Competition:** Our literature review shows that most previous research uses single-agent techniques. Conversely, real-world computation offloading environments are often multi-agent. They include multiple BSs, multiple smart gateways, multiple edge/fog servers, or multiple user devices. In such multi-agent settings, the reward an agent receives depends on both its actions and the actions of others. Also, convergence may be slow and unstable, because all agents learn in parallel. One of the open issues is designing computation offloading scenarios for multi-agent environments. These scenarios may be cooperative [3, 10, 194] or competitive [117]. Some research [6] suggests grouping methods to prevent overestimation. However, the exchange of information between agents may increase the signaling overhead. One of the future trends is the combination of DRL with game theory and federated learning for more complex scenarios.
- **Partially Observable Environments:** Unfortunately, computation offloading environments are not always fully observable. Limited environmental information is available in the Partially Observable MDP. They provide only partial information instead of complete information. One way to overcome this problem is to keep track of previous state information in memory to understand the nature of the environment [10]. Given recent advances in computing power, especially quantum computing, NNs are expected to be a promising solution for implementing POMDP.

Due to space limitations, the continuation of this section is transferred to Appendix A.4. Also, interested readers can refer to Reference [220] for a deeper study.

7 CONCLUSION AND FUTURE TRENDS

This article reviewed current research on RL techniques in computation offloading. After describing the advantages and disadvantages of each RL technique, we analyzed related research in both binary and partial offloading modes. This analysis was performed based on factors such as the type of RL algorithm, offloading mode, performance metrics, MDP components, single-agent/multi-agent environment, implementation issues, and core ideas. Finally, the survey answered eight fundamental questions. It revealed that the DQN method, with an approximate share of 22%, and the Q-Learning with 13%, were the most popular RL techniques. Among on-policy algorithms, the percentage of AC was higher than the rest; 72% of the studies used the binary offloading scheme, and the remaining 28% used a partial one. In most RL research, performance metrics such as latency,

energy consumption, reward, and cost have attracted the most attention from researchers. Approximately 38% of the papers did not mention their evaluation toolkit; 18% used Python and Tensorflow tools, 8% used Tensorflow, 8% Python, and 7% MATLAB. Most RL-based research has been conducted on the MEC paradigm. After discussing open issues, four areas were ultimately explored as future research trends. These areas include the curse of dimensionality, discretization of the state-action space, multi-agent cooperation/competition, and partially observable environments.

REFERENCES

- [1] Ibrahim A. Elgendy, Wei-Zhe Zhang, Hui He, Brij B. Gupta, and Ahmed A. Abd El-Latif. 2021. Joint computation offloading and task caching for multi-user and multi-task MEC systems: Reinforcement learning-based algorithms. *Wireless Netw.* 27 (2021), 2023–2038. <https://doi.org/10.1007/s11276-021-02554-w>
- [2] Guisong Yang, Ling Hou, Hao Cheng, Xingyu He, Daojing He, and Sammy Chan. 2020. Computation offloading time optimization via Q-learning in opportunistic edge computing. *IET Commun.* 14 (2020), 3898–3906. <https://doi.org/10.1049/iet-com.2020.0765>
- [3] Xing Chen and Guizhong Liu. 2021. Energy-Efficient task offloading and resource allocation via deep reinforcement learning for augmented reality in mobile edge networks. *IEEE Internet Things J.* 8 (2021), 10843–10856. <https://doi.org/10.1109/jiot.2021.3050804>
- [4] Vahid Jafari and Mohammad Hossein Rezvani. 2021. Joint optimization of energy consumption and time delay in IoT-fog-cloud computing environments using NSGA-II metaheuristic algorithm. *J. Ambient Intell. Human. Comput.* (2021), 1–24. <https://doi.org/10.1007/s12652-021-03388-2>
- [5] G. M. Shafiqur Rahman, Tian Dang, and Manzoor Ahmed. 2020. Deep reinforcement learning based computation offloading and resource allocation for low-latency fog radio access networks. *Intell. Conver. Netw.* 1 (2020), 243–257. <https://doi.org/10.23919/icn.2020.0020>
- [6] Miaojiang Chen, Tian Wang, Shaobo Zhang, and Anfeng Liu. 2021. Deep reinforcement learning for computation offloading in mobile edge computing environment. *Computer Commun.* 175 (2021), 1–12. <https://doi.org/10.1016/j.comcom.2021.04.028>
- [7] Jia Yan, Suzhi Bi, and Ying-Jun Angela Zhang. 2020. Offloading and resource allocation with general task graph in mobile edge computing: A deep reinforcement learning approach. *IEEE Trans. Wireless Commun.* 19, 8 (2020), 5404–5419. <https://doi.org/10.1109/TWC.2020.2993071>
- [8] Taha Alfakih, Mohammad Mehdi Hassan, Abdu Gumaei, Claudio Savaglio, and Giancarlo Fortino. 2020. Task offloading and resource allocation for mobile edge computing by deep reinforcement learning based on SARSA. *IEEE Access* 8 (2020), 54074–54084. <https://doi.org/10.1109/access.2020.2981434>
- [9] Samrat Nath and Jingxian Wu. 2020. Deep reinforcement learning for dynamic computation offloading and resource allocation in cache-assisted mobile edge computing systems. *Intell. Conver. Netw.* 1 (2020), 181–198. <https://doi.org/10.23919/icn.2020.0014>
- [10] Jungyeon Baek and Georges Kaddoum. 2021. Heterogeneous task offloading and resource allocations via deep recurrent reinforcement learning in partial observable multifog networks. *IEEE Internet Things J.* 8 (2021), 1041–1056. <https://doi.org/10.1109/jiot.2020.3009540>
- [11] Maryam Keshavarznejad, Mohammad Hossein Rezvani, and Sepideh Adabi. 2021. Delay-aware optimization of energy consumption for task offloading in fog environments using metaheuristic algorithms. *Cluster Comput.* 24 (2021), 1825–1853. <https://doi.org/10.1007/s10586-020-03230-y>
- [12] Flavio Bonomi, Rodolfo Milito, Jiang Zhu, and Sateesh Addepalli. 2012. Fog computing and its role in the internet of things. In *Proceedings of the 1st Edition of the MCC Workshop on Mobile Cloud Computing (MCC'12)*. <https://doi.org/10.1145/2342509.2342513>
- [13] Mohammad Hassan Khoobkar, Mehdi Dehghan Takht Fooladi, Mohammad Hossein Rezvani, and Mohammad Mehdi Gilanian Sadeghi. 2022. Partial offloading with stable equilibrium in fog-cloud environments using replicator dynamics of evolutionary game theory. *Cluster Comput.* 25 (2022), 1393–1420.
- [14] Akhirul Islam, Arindam Deb Nath, Manojit Ghose, and Suchetana Chakraborty. 2021. A Survey on Task Offloading in Multi-access Edge Computing. *J. Syst. Architect.* 118 (2021), 102225. <https://doi.org/10.1016/j.sysarc.2021.102225>
- [15] Haifeng Lu, Chunhua Gu, Fei Luo, Weichao Ding, and Xinpeng Liu. 2020. Optimization of lightweight task offloading strategy for mobile edge computing based on deep reinforcement learning. *Future Gen. Comput. Syst.* 102 (2020), 847–861. <https://doi.org/10.1016/j.future.2019.07.019>
- [16] Wenhan Zhan, Chunbo Luo, Jin Wang, Geyong Min, and Hancong Duan. 2019. Deep reinforcement learning-based computation offloading in vehicular edge computing. In *Proceedings of the IEEE Global Communications Conference (GLOBECOM'19)*. <https://doi.org/10.1109/globecom38437.2019.9013982>

- [17] Richard S. Sutton and Andrew G. Barto. 2018. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, MA.
- [18] Liu Jinshi, Manzoor Ahmed, Muhammad Ayzed Mirza, Wali Ullah Khan, Dianlei Xu, Jianbo Li, Abdul Aziz, and Zhu Han. 2022. RL/DRL meets vehicular task offloading using edge and vehicular cloudlet: A survey. *IEEE Internet Things J.* 9, 11 (2022), 8315–8338. <https://doi.org/10.1109/JIOT.2022.3155667>
- [19] Arash Heidari, Mohammad Ali Jabraeil Jamali, Nima Jafari Navimipour, and Shahin Akbarpour. 2020. Internet of Things offloading: Ongoing issues, opportunities, and future challenges. *Int. J. Commun. Syst.* 33 (2020). <https://doi.org/10.1002/dac.4474>
- [20] Sardar Khaliq uz Zaman, Ali Imran Jehangiri, Tahir Maqsood, Zulfiqar Ahmad, Arif Iqbal Umar, Junaid Shuja, Eisa Alanazi, and Waleed Alasmay. 2021. Mobility-aware computational offloading in mobile edge networks: A survey. *Cluster Comput.* 24 (2021), 2735–2756. <https://doi.org/10.1007/s10586-021-03268-6>
- [21] Firdose Saeik, Marios Avgeris, Dimitrios Spatharakis, Nina Santi, Dimitrios Dechouniotis, John Violos, Aris Leivadeas, Nikolaos Athanasopoulos, Nathalie Mitton, and Symeon Papavassiliou. 2021. Task offloading in Edge and Cloud Computing: A survey on mathematical, artificial intelligence and control theory solutions. *Comput. Netw.* 195 (2021), 108177. <https://doi.org/10.1016/j.comnet.2021.108177>
- [22] Jianyu Wang, Jianli Pan, Flavio Esposito, Prasad Calyam, Zhicheng Yang, and Prasant Mohapatra. 2020. Edge cloud offloading algorithms. *ACM Comput. Surveys* 52 (2020), 1–23. <https://doi.org/10.1145/3284387>
- [23] Mohammad Masdari and Hemn Khezri. 2020. Efficient offloading schemes using Markovian models: A literature review. *Computing* 102 (2020), 1673–1716. <https://doi.org/10.1007/s00607-020-00812-x>
- [24] Ali Shakarami, Ali Shahidinejad, and Mostafa Ghobaei-Arani. 2020. A review on the computation offloading approaches in mobile edge computing: A game-theoretic perspective. *Softw.: Pract. Exper.* 50, 1719–1759. <https://doi.org/10.1002/spe.2839>
- [25] Ali Shakarami, Mostafa Ghobaei-Arani, and Ali Shahidinejad. 2020. A survey on the computation offloading approaches in mobile edge computing: A machine learning-based perspective. *Comput. Netw.* 182 (2020), 107496. <https://doi.org/10.1016/j.comnet.2020.107496>
- [26] Amir Masoud Rahmani, Mokhtar Mohammadi, Adil Hussein Mohammed, Sarkhel H. Taher Karim, Mohammed Kamal Majeed, Mohammed Masdari, and Mehdi Hosseinzadeh. 2021. Towards data and computation offloading in mobile cloud computing: Taxonomy, overview, and future directions. *Wireless Personal Commun.* 119 (2021), 147–185. <https://doi.org/10.1007/s11277-021-08202-y>
- [27] Azzedine Boukerche, Shichao Guan, and Robson E. De Grande. 2019. Sustainable offloading in mobile cloud computing. *ACM Comput. Surveys* 52 (2019), 1–37. <https://doi.org/10.1145/3286688>
- [28] Kaouther Gasmı, Selma Dilek, Suleyman Tosun, and Suat Ozdemir. 2021. A survey on computation offloading and service placement in fog computing-based IoT. *J. Supercomput.* 78 (2021), 1983–2014. <https://doi.org/10.1007/s11227-021-03941-y>
- [29] Alisson Barbosa De Souza, Paulo A. L. Rego, Tilago Carneiro, Jardel Das C. Rodrigues, Pedro Pedrosa Rebouças Filho, José Neuman De Souza, Vinay Chamola, Victor Hugo C. De Albuquerque, and Bipab Sikdar. 2020. Computation offloading for vehicular environments: A survey. *IEEE Access* 8 (2020), 198214–198243. <https://doi.org/10.1109/ACCESS.2020.3033828>
- [30] Hai Lin, Sherali Zeadally, Zhihong Chen, Houda Labiod, and Lusheng Wang. 2020. A survey on computation offloading modeling for edge computing. *J. Netw. Comput. Appl.* 169 (2020), 102781. <https://doi.org/10.1016/j.jnca.2020.102781>
- [31] Congfeng Jiang, Xiaolan Cheng, Honghao Gao, Xin Zhou, and Jian Wan. 2019. Toward computation offloading in edge computing: A survey. *IEEE Access* 7 (2019), 131543–131558. <https://doi.org/10.1109/access.2019.2938660>
- [32] Ali Shakarami, Mostafa Ghobaei-Arani, Mohammad Masdari, and Mehdi Hosseinzadeh. 2020. A survey on the computation offloading approaches in mobile edge/cloud computing environment: A stochastic-based perspective. *J. Grid Comput.* 18 (2020), 639–671. <https://doi.org/10.1007/s10723-020-09530-2>
- [33] Nguyen Cong Luong, Dinh Thai Hoang, Shimin Gong, Dusit Niyato, Ping Wang, Ying-Chang Liang, and Dong in Kim. 2019. Applications of deep reinforcement learning in communications and networking: A survey. *IEEE Commun. Surveys Tutor.* 21, 4 (2019), 3133–3174. <https://doi.org/10.1109/COMST.2019.2916583>
- [34] Lei Lei, Yue Tan, Kan Zheng, Shiwen Liu, Kuan Zhang, and Xuemin Shen. 2020. Deep reinforcement learning for autonomous internet of things: Model, applications and challenges. *IEEE Commun. Surveys Tutor.* 22, 3 (2020), 1722–1760. <https://doi.org/10.1109/COMST.2020.2988367>
- [35] Li Tianxu, Kun Zhu, Nguyen Cong Luong, Dusit Niyato, Qihui Wu, Yang Zhang, and Bing Chen. 2022. Applications of multi-agent reinforcement learning in future internet: A comprehensive survey. *IEEE Commun. Surveys Tutor.* (2022). <https://doi.org/10.1109/COMST.2022.3160697>
- [36] Tran-Dang Hoa, Sanjay Bhardwaj, Tariq Rahim, Arslan Musaddiq, and Dong-Seong Kim. 2022. Reinforcement learning based resource management for fog computing environment: Literature review, challenges, and open issues. *J. Commun. Netw.* (2022). <https://doi.org/10.23919/JCN.2021.000041>

- [37] P. Mell and T. Grance. 2011. *The NIST Definition of Cloud Computing*. Computer Security Division, Information Technology Laboratory National Institute of Standards and Technology, Gaithersburg, MD.
- [38] Hoang T. Dinh, Chonho Lee, Dusit Niyato, and Ping Wang. 2011. A survey of mobile cloud computing: architecture, applications, and approaches. *Wireless Commun. Mobile Comput.* 13 (2011), 1587–1611. <https://doi.org/10.1002/wcm.1203>
- [39] Nirosinie Fernando, Seng W. Loke, and Wenny Rahayu. 2013. Mobile cloud computing: A survey. *Future Gen. Comput. Syst.* 29, 1 (2013), 84–106. <https://doi.org/10.1016/j.future.2012.05.023>
- [40] Pavel Mach and Zdenek Becvar. 2017. Mobile edge computing: A survey on architecture and computation offloading. *IEEE Commun. Surveys Tutor.* 19, 3 (2017), 1628–1656. <https://doi.org/10.1109/COMST.2017.2682318>
- [41] Yuyi Mao, Changsheng You, Jun Zhang, Kaibin Huang, and Khaled B. Letaief. 2017. A survey on mobile edge computing: The communication perspective. *IEEE Commun. Surveys Tutor.* 19 (2017), 2322–2358. <https://doi.org/10.1109/comst.2017.2745201>
- [42] Waheed Abdul, Munam Ali Shah, Syed Muhammad Mohsin, Abid Khan, Carsten Maple, Sheraz Aslam, and Shahab Shamshirband. 2022. A comprehensive review of computing paradigms, enabling computation offloading and task execution in vehicular networks. *IEEE Access* (2022). <https://doi.org/10.1109/ACCESS.2021.3138219>
- [43] OpenFog Consortium Architecture Working Group. 2017. OpenFog Reference Architecture for Fog Computing, *Technical Report*. Retrieved from https://www.iiconsortium.org/pdf/OpenFog_Reference_Architecture_2_09_17.pdf.
- [44] Wang Mingwei, Hualing Yi, Feng Jiang, Ling Lin, and Min Gao. 2022. Review on offloading of vehicle edge computing. *J. Artif. Intell. Technol.* 2, 4 (2022), 132–143. <https://doi.org/10.37965/jait.2022.0120>
- [45] Hamdi Aisha Muhammad A., Farookh Khadeer Hussain, and Omar K. Hussain. 2022. Task offloading in vehicular fog computing: State-of-the-art and open issues. *Future Gen. Comput. Syst.* <https://doi.org/10.1016/j.future.2022.03.019>
- [46] Liang Huang, Xu Feng, Cheng Zhang, Liping Qian, and Yuan Wu. 2019. Deep reinforcement learning-based joint task offloading and bandwidth allocation for multi-user mobile edge computing. *Dig. Commun. Netw.* 5 (2019), 10–17. <https://doi.org/10.1016/j.dcan.2018.10.003>
- [47] Kaiyi Zhang. 2020. *Task Offloading and Resource Allocation Using Deep Reinforcement Learning*. Doctoral dissertation, Université d'Ottawa/University of Ottawa. <http://dx.doi.org/10.20381/ruor-25749>
- [48] S. Pan, Z. Zhang, Z. Zhang, and D. Zeng. 2019. Dependency-aware computation offloading in mobile edge computing: A reinforcement learning approach. *IEEE Access* 7 (2019), 134742–134753.
- [49] Lixiong Leng, Jingchen Li, Haobin Shi, and Yi'an Zhu. 2021. Graph convolutional network-based reinforcement learning for tasks offloading in multi-access edge computing. *Multimedia Tools Appl.* 80 (2021), 29163–29175. <https://doi.org/10.1007/s11042-021-11130-5>
- [50] Leslie Pack Kaelbling, Michael L. Littman, and Andrew W. Moore. 1996. Reinforcement learning: A survey. *J. Artif. Intell. Res.* 4 (1996), 237–285. <https://doi.org/10.1613/jair.301>
- [51] Christopher John Cornish Hellaby Watkins. 1989. *Learning from Delayed Rewards*. Cambridge University Press, Cambridge, UK.
- [52] Christopher J. C. H. Watkins and Peter Dayan. 1992. Q-learning. *Mach. Learn.* 8, 3 (1992), 279–292. <https://doi.org/10.1007/BF00992698>
- [53] Haifeng Lu, Chunhua Gu, Fei Luo, Weichao Ding, Shuai Zheng, and Yifan Shen. 2020. Optimization of task offloading strategy for mobile edge computing based on multi-agent deep reinforcement learning. *IEEE Access* 8 (2020), 202573–202584. <https://doi.org/10.1109/access.2020.3036416>
- [54] Yaohua Sun, Mugen Peng, Yangcheng Zhou, Yuzhe Huang, and Shiwen Mao. 2019. Application of machine learning in wireless networks: Key techniques and open issues. *IEEE Commun. Surveys Tutor.* 21 (2019), 3072–3108. <https://doi.org/10.1109/comst.2019.2924243>
- [55] Richard Bellman. 1957. A Markovian decision processes. *J. Math. Mech.* 6, 5 (1957), 679–684. <http://www.jstor.org/stable/24900506>.
- [56] Xiaofei Wang, Yiwen Han, Victor C. M. Leung, Dusit Niyato, Xueqiang Yan, and Xu Chen. 2020. Convergence of edge computing and deep learning: A comprehensive survey. *IEEE Commun. Surveys Tutor.* 22, 2 (2020), 869–904. <https://doi.org/10.1109/COMST.2020.2970550>
- [57] Yuxi Li. 2018. Deep reinforcement learning: An overview. Retrieved from <https://arXiv:1701.07274>. <https://doi.org/10.48550/arXiv.1701.07274>
- [58] Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. 1998. Planning and acting in partially observable stochastic domains. *Artificial Intell.* 101, 1-2 (1998), 99–134. DOI: [https://doi.org/10.1016/S0004-3702\(98\)00023-X](https://doi.org/10.1016/S0004-3702(98)00023-X)
- [59] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. 2016. *Deep Learning*. MIT Press, Cambridge, MA.
- [60] Vincent François-Lavet, Peter Henderson, Riashat Islam, Marc G. Bellemare, and Joelle Pineau. 2018. An introduction to deep reinforcement learning. *Found. Trends Mach. Learn.* 11, 3-4 (2018). <https://doi.org/10.1561/22000000071>
- [61] Kai Arulkumaran, Marc Peter Deisenroth, Miles Brundage, and Anil Anthony Bharath. 2017. A brief survey of deep reinforcement learning. Retrieved from <https://arXiv:1708.05866>. <https://doi.org/10.48550/arXiv.1708.05866>

- [62] Mohit Sewak. 2019. *Deep Reinforcement Learning Frontiers of Artificial Intelligence*. Springer, Singapore. <https://doi.org/10.1007/978-981-13-8285-7>
- [63] Hao Dong, Zihan Ding, and Shanghang Zhang. 2020. *Deep Reinforcement Learning: Fundamentals, Research and Applications*. Springer, Singapore. <https://doi.org/10.1007/978-981-15-4095-0>
- [64] Gerhard Weiss (Ed.). 1999. *Multiagent Systems: A Modern Approach to Distributed Artificial Intelligence*. MIT Press, Cambridge, MA.
- [65] Ming Tan. 1993. Multi-agent reinforcement learning: Independent vs. cooperative agents. *Mach. Learn. Proc.* (1993), 330–337. <https://doi.org/10.1016/b978-1-55860-307-3.50049-6>.
- [66] Lucian Buşoniu, Robert Babuška, and Bart De Schutter. 2008. A comprehensive survey of multiagent reinforcement learning. *IEEE Trans. Syst., Man Cybernet., Part C (Appl. Rev.)* 38, 2 (2008), 156–172. <https://doi.org/10.1109/TSMCC.2007.913919>
- [67] Liviu Panait and Sean Luke. 2005. Cooperative multi-agent learning: The state of the art. *Autonomous Agents Multi-Agent Syst.* 11 (2005), 387–434. <https://doi.org/10.1007/s10458-005-2631-2>
- [68] Kaiqing Zhang, Zhuoran Yang, and Tamer Başar. 2021. Multi-agent reinforcement learning: A selective overview of theories and algorithms. handbook reinforce. *Learn. Control* (2021), 321–384. https://doi.org/10.1007/978-3-030-60990-0_12
- [69] Pablo Hernandez-Leal, Bilal Kartal, and Matthew E. Taylor. 2019. A survey and critique of multiagent deep reinforcement learning. *Auton. Agents Multi-Agent Syst.* 33, 6 (2019), 750–797. <https://doi.org/10.1007/s10458-019-09421-1>
- [70] Barbara Kitchenham, O. Pearl Brereton, David Budgen, Mark Turner, John Bailey, and Stephen Linkman. 2009. Systematic literature reviews in software engineering—A systematic literature review. *Info. Softw. Technol.* 51 (2009), 7–15. <https://doi.org/10.1016/j.infsof.2008.09.009>
- [71] Miao Yang, Hongbin Zhu, Haifeng Wang, Yevgeni Koucheryavy, Konstantin Samouylov, and Hua Qian. 2021. An online learning approach to computation offloading in dynamic fog networks. *IEEE Internet Things J.* 8 (2021), 1572–1584. <https://doi.org/10.1109/jiot.2020.3015522>
- [72] Chen Chen, Yuru Zhang, Zheng Wang, Shaohua Wan, and Qingqi Pei. 2021. Distributed computation offloading method based on deep reinforcement learning in ICV. *Appl. Soft Comput.* 103 (2021), 107108. <https://doi.org/10.1016/j.asoc.2021.107108>
- [73] Manoj Penmetcha and Byung-Cheol Min. 2021. A deep reinforcement learning-based dynamic computational offloading method for cloud robotics. *IEEE Access* 9 (2021), 60265–60279. <https://doi.org/10.1109/access.2021.3073902>
- [74] Xiaoyu Zhu, Yueyi Luo, Anfeng Liu, Md Zakirul Alam Bhuiyan, and Shaobo Zhang. 2021. Multiagent deep reinforcement learning for vehicular computation offloading in IoT. *IEEE Internet Things J.* 8 (2021), 9763–9773. <https://doi.org/10.1109/jiot.2020.3040768>
- [75] Jin Wang, Jia Hu, Geyong Min, Wenhan Zhan, Qiang Ni, and Nektarios Georgalas. 2019. Computation offloading in multi-access edge computing using a deep sequential model based on reinforcement learning. *IEEE Commun. Mag.* 57 (2019), 64–69. <https://doi.org/10.1109/mcom.2019.1800971>
- [76] M. S. Mekala, Alireza Jolfaei, Gautam Srivastava, Xi Zheng, Amjad Anvari-Moghaddam, and P. Viswanathan. 2022. Resource offload consolidation based on deep-reinforcement learning approach in cyber-physical systems. *IEEE Trans. Emerg. Topics Comput. Intell.* 6 (2022), 245–254.
- [77] Ahsan Rafiq, Wang Ping, Wei Min, and Mohammad Saleh Ali Muthanna. 2021. Fog Assisted 6TiSCH Tri-Layer network architecture for adaptive scheduling and energy-efficient offloading using rank-based Q-Learning in smart industries. *IEEE Sensors J.* 21 (2021), 25489–25507.
- [78] Soohyun Park, Dohyun Kwon, Joongheon Kim, Younkyu Lee, and Sungrae Cho. 2020. Adaptive Real-Time offloading decision-making for mobile edges: Deep reinforcement learning framework and simulation results. *Appl. Sci.* 10 (2020), 1663. <https://doi.org/10.3390/app10051663>
- [79] Ning Chen, Sheng Zhang, Zhuzhong Qian, Jie Wu, and Sanglu Lu. 2019. When learning joins edge: Real-Time proportional computation offloading via deep reinforcement learning. In *Proceedings of the IEEE 25th International Conference on Parallel and Distributed Systems (ICPADS'19)*. <https://doi.org/10.1109/icpads47876.2019.00066>
- [80] H. C. Ke, H. Wang, H. W. Zhao, and W. J. Sun. 2021. Deep reinforcement learning-based computation offloading and resource allocation in security-aware mobile edge computing. *Wireless Netw.* 27 (2021), 3357–3373. <https://doi.org/10.1007/s11276-021-02643-w>
- [81] Hui Wang, Hongchang Ke, and Weijia Sun. 2020. Unmanned-Aerial-Vehicle-Assisted computation offloading for mobile edge computing based on deep reinforcement learning. *IEEE Access* 8 (2020), 180784–180798. <https://doi.org/10.1109/access.2020.3028553>
- [82] Yueyue Dai, Ke Zhang, Sabita Maharjan, and Yan Zhang. 2021. Deep reinforcement learning for stochastic computation offloading in digital twin networks. *IEEE Trans. Industr. Inform.* 17 (2021), 4968–4977. <https://doi.org/10.1109/tii.2020.3016320>

- [83] Jianbo Du, F. Richard Yu, Guangyue Lu, Junxuan Wang, Jing Jiang, and Xiaoli Chu. 2020. MEC-assisted immersive vr video streaming over terahertz wireless networks: A deep reinforcement learning approach. *IEEE Internet Things J.* 7 (2020), 9517–9529. <https://doi.org/10.1109/jiot.2020.3003449>
- [84] Shimin Gong, Yutong Xie, Jing Xu, Dusit Niyato, and Ying-Chang Liang. 2020. Deep reinforcement learning for backscatter-aided data offloading in mobile edge computing. *IEEE Netw.* 34 (2020), 106–113. <https://doi.org/10.1109/mnet.001.1900561>
- [85] Yijing Ren, Yaohua Sun, and Mugen Peng. 2021. Deep reinforcement learning based computation offloading in fog enabled industrial internet of things. *IEEE Trans. Industr. Inform.* 17 (2021), 4978–4987. <https://doi.org/10.1109/tii.2020.3021024>
- [86] Hongchang Ke, Jian Wang, Lingyue Deng, Yuming Ge, and Hui Wang. 2020. Deep reinforcement learning-based adaptive computation offloading for MEC in heterogeneous vehicular networks. *IEEE Trans. Vehic. Technol.* 69 (2020), 7916–7929. <https://doi.org/10.1109/tvt.2020.2993849>
- [87] Hongchang Ke, Jian Wang, Hui Wang, and Yuming Ge. 2019. Joint optimization of data offloading and resource allocation with renewable energy aware for IoT devices: A deep reinforcement learning approach. *IEEE Access* 7 (2019), 179349–179363. <https://doi.org/10.1109/access.2019.2959348>
- [88] Dinh C. Nguyen, Pubudu N. Pathirana, Ming Ding, and Aruna Seneviratne. 2020. Privacy-Preserved task offloading in mobile blockchain with deep reinforcement learning. *IEEE Trans. Netw. Serv. Manag.* 17 (2020), 2536–2549. <https://doi.org/10.1109/tnsm.2020.3010967>
- [89] Ji Li, Hui Gao, Tiejun Lv, and Yueming Lu. 2018. Deep reinforcement learning based computation offloading and resource allocation for MEC. In *Proceedings of the IEEE Wireless Communications and Networking Conference (WCNC'18)*. <https://doi.org/10.1109/wcnc.2018.8377343>
- [90] HaiZhong Tan and Limin Zhu. 2020. Overall computing offloading strategy based on deep reinforcement learning in vehicle fog computing. *J. Eng.* (2020), 1080–1087. <https://doi.org/10.1049/joe.2020.0134>
- [91] Jing Zhang, Jun Du, Yuan Shen, and Jian Wang. 2020. Dynamic computation offloading with energy harvesting devices: A hybrid-decision-based deep reinforcement learning approach. *IEEE Internet Things J.* 7 (2020), 9303–9317. <https://doi.org/10.1109/jiot.2020.3000527>
- [92] Zhao Chen and Xiaodong Wang. 2020. Decentralized computation offloading for multi-user mobile edge computing: A deep reinforcement learning approach. *EURASIP J. Wireless Commun. Netw.* (2020). <https://doi.org/10.1186/s13638-020-01801-6>
- [93] Yongkang Gong, Jingjing Wang, and Tianzheng Nie. 2020. Deep reinforcement learning aided computation offloading and resource allocation for IoT. In *Proceedings of the IEEE Computing, Communications and IoT Applications Conference (ComComAp'20)*. <https://doi.org/10.1109/comcomap51192.2020.9398891>
- [94] Kun Wang, Xiaofeng Wang, Xuan Liu, and Alireza Jolfaei. 2020. Task offloading strategy based on reinforcement learning computing in edge computing architecture of internet of vehicles. *IEEE Access* 8 (2020), 173779–173789. <https://doi.org/10.1109/access.2020.3023939>
- [95] Zhuo Li, Xu Zhou, Taixin Li, and Yang Liu. 2021. An optimal-transport-based reinforcement learning approach for computation offloading. In *Proceedings of the IEEE Wireless Communications and Networking Conference (WCNC'21)*. <https://doi.org/10.1109/wcnc49053.2021.9417331>
- [96] Dali Zhu, Haitao Liu, Ting Li, Jiyan Sun, Jie Liang, Hangsheng Zhang, Liru Geng, and Yinlong Liu. 2021. Deep reinforcement learning-based task offloading in satellite-terrestrial edge computing networks. In *Proceedings of the IEEE Wireless Communications and Networking Conference (WCNC'21)*. <https://doi.org/10.1109/wcnc49053.2021.9417127>
- [97] Xiongwen Cheng, Feng Lyu, Wei Quan, Conghao Zhou, Hongli He, Weisen Shi, and Xuemin Shen. 2019. Space/aerial-assisted computing offloading for IoT applications: A learning-based approach. *IEEE J. Select. Areas Commun.* 37 (2019), 1117–1129. <https://doi.org/10.1109/jsac.2019.2906789>
- [98] Kaiyi Zhang and Nancy Samaan. 2020. Optimized look-ahead offloading decisions using deep reinforcement learning for battery constrained mobile IoT devices. In *Proceedings of the IEEE International Conference on Smart Cloud (SmartCloud'20)*. <https://doi.org/10.1109/smartcloud49737.2020.00042>
- [99] Cheng Zhang, Zhi Liu, Bo Gu, Kyoko Yamori, and Yoshiaki Tanaka. 2018. A deep reinforcement learning based approach for cost- and energy-aware multi-flow mobile data offloading. *IEICE Trans. Commun.* (2018), 1625–1634. <https://doi.org/10.1587/transcom.2017cqp0014>
- [100] Jie Xu, Lixing Chen, and Shaolei Ren. 2017. Online learning for offloading and autoscaling in energy harvesting mobile edge computing. *IEEE Trans. Cogn. Commun. Netw.* 3 (2017), 361–373. <https://doi.org/10.1109/tccn.2017.2725277>
- [101] Mona Naghdehforoushha, Mehdi Dehghan Takht Fooladi, Mohammad Hossein Rezvani, and Mohammad Mehdi Gilanian Sadeghi. 2022. BLMDP: A new bi-level Markov decision process approach to joint bidding and task-scheduling in cloud spot market. *Turkish J. Electr. Eng. Comput. Sci.* 30, 4 (2022), 1419–1438. <https://doi.org/10.55730/1300-0632.3857>

- [102] Peter Auer, Nicolò Cesa-Bianchi, and Paul Fischer. 2002. Finite-time analysis of the multiarmed bandit problem. *Mach. Learn.* 47, 2 (2002), 235–256. <https://doi.org/10.1023/A:1013689704352>
- [103] Donald A. Berry and Bert Fristedt. 1985. *Bandit Problems: Sequential Allocation of Experiments (Monographs on Statistics and Applied Probability)*. Chapman & Hall, London, UK. <https://doi.org/10.1007/978-94-015-3711-7>
- [104] Sébastien Bubeck and Nicolo Cesa-Bianchi. 2012. Regret analysis of stochastic and nonstochastic multi-armed bandit problems. Retrieved from <https://arXiv:1204.5721>.
- [105] Xin Gao, Xi Huang, Ziyu Shao, and Yang Yang. 2021. An integration of online learning and online control for green offloading in fog-assisted IoT systems. *IEEE Trans. Green Commun. Network.* 5 (2021), 1632–1646. <https://doi.org/10.1109/tgcn.2021.3083426>
- [106] Xueying Zhang, Ruiting Zhou, Zhi Zhou, John C. S. C. S. Lui, and Zongpeng Li. 2020. An online learning-based task offloading framework for 5g small cell networks. In *Proceedings of the 49th International Conference on Parallel Processing (ICPP'20)*. <https://doi.org/10.1145/3404397.3404417>
- [107] Gavin Adrian Rummery and Mehesan Niranjan. 1994. *On-Line Q-Learning Using Connectionist Systems*. University of Cambridge Department of Engineering, Cambridge, UK.
- [108] Xianfu Chen, Honggang Zhang, Celimuge Wu, Shiwen Mao, Yusheng Ji, and Medhi Bennis. 2019. Optimized computation offloading performance in virtual edge computing systems via deep reinforcement learning. *IEEE Internet Things J.* 6 (2019), 4005–4018. <https://doi.org/10.1109/jiot.2018.2876279>
- [109] Satish Vemireddy and Rashmi Ranjan Rout. 2021. Fuzzy reinforcement learning for energy efficient task offloading in vehicular fog computing. *Comput. Netw.* 199 (2021), 108463. <https://doi.org/10.1016/j.comnet.2021.108463>
- [110] Hui Huang, Qiang Ye, and Hongwei Du. 2020. Reinforcement learning based offloading for realtime applications in mobile edge computing. In *Proceedings of the IEEE International Conference on Communications (ICC'20)*. <https://doi.org/10.1109/icc40277.2020.9148748>
- [111] Liqiang Zhang, Jingjing Luo, Lin Gao, and Fu-Chun Zheng. 2020. Learning-based computation offloading for edge networks with heterogeneous resources. In *Proceedings of the IEEE International Conference on Communications (ICC'20)*. <https://doi.org/10.1109/icc40277.2020.9149171>
- [112] Xuezhu Li. 2021. A computing offloading resource allocation scheme using deep reinforcement learning in mobile edge computing systems. *J. Grid Comput.* 19 (2021). <https://doi.org/10.1007/s10723-021-09568-w>
- [113] Kun Wang, Xiaofeng Wang, and Xuan Liu. 2021. A high reliable computing offloading strategy using deep reinforcement learning for IoVs in edge computing. *J. Grid Comput.* 19 (2021). <https://doi.org/10.1007/s10723-021-09542-6>
- [114] Meng Wang, Shuo Shi, Shushi Gu, Xuemai Gu, and Xue Qin. 2020. Q-learning based computation offloading for multi-UAV-enabled cloud-edge computing networks. *IET Commun.* 14 (2020), 2481–2490. <https://doi.org/10.1049/iet-com.2019.1184>
- [115] Yuepeng Li, Lvhao Chen, Deze Zeng, and Lin Gu. 2020. A customized reinforcement learning based binary offloading in edge cloud. In *Proceedings of the IEEE 26th International Conference on Parallel and Distributed Systems (ICPADS'20)*. <https://doi.org/10.1109/icpads51040.2020.00055>
- [116] Peng Liu, Han He, Tingting Fu, Huijuan Lu, Abdulhameed Alelaiwi, and Md Wasif Islam Wasi. 2021. Task offloading optimization of cruising UAV with fixed trajectory. *Comput. Netw.* 199 (2021), 108397. <https://doi.org/10.1016/j.comnet.2021.108397>
- [117] Jiangyi Hu, Yang Li, Gaofeng Zhao, Bo Xu, Yiyang Ni, and Haitao Zhao. 2021. Deep reinforcement learning for task offloading in edge computing assisted power IoT. *IEEE Access* 9 (2021), 93892–93901. <https://doi.org/10.1109/access.2021.3092381>
- [118] Emna Fakhfakh and Soumaya Hamouda. 2017. Optimised Q-learning for WiFi offloading in dense cellular networks. *IET Commun.* 11 (2017), 2380–2385.
- [119] Emna Fakhfakh and Soumaya Hamouda. 2017. Incentive reward for efficient WiFi offloading using Q-learning approach. In *Proceedings of the 13th International Wireless Communications and Mobile Computing Conference (IWCMC'17)*. <https://doi.org/10.1109/iwcmc.2017.7986441>
- [120] Hado van Hasselt, Arthur Guez, and David Silver. 2016. Deep reinforcement learning with double q-learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*. DOI: <https://doi.org/10.1609/aaai.v30i1.10295>
- [121] Volodymyr Mnih et al. 2015. Human-level control through deep reinforcement learning. *Nature* 518, 7540 (2015), 529–533. DOI: <https://doi.org/10.1038/nature14236>
- [122] Sudharsan Ravichandiran. 2020. *Deep Reinforcement Learning with Python*. Packt Publishing.
- [123] Tom Schaul, John Quan, Ioannis Antonoglou and David Silver. 2015. Prioritized experience replay. Retrieved from <https://arXiv:1511.05952>. <https://doi.org/10.48550/arXiv.1511.05952>
- [124] Shuo Shi, Meng Wang, Shushi Gu, and Zhong Zheng. 2021. Energy-efficient UAV-enabled computation offloading for industrial internet of things: A deep reinforcement learning approach. *Wireless Netw.* <https://doi.org/10.1007/s11276-021-02789-7>

- [125] Gaibin Li, Mingkai Chen, Xin Wei, Ting Qi, and Wenqin Zhuang. 2020. Computation offloading with reinforcement learning in D2D-MEC network. In *Proceedings of the International Wireless Communications and Mobile Computing (IWCMC'20)*. <https://doi.org/10.1109/iwcmc48107.2020.9148285>
- [126] Jinkyoo Jeong, Il-Min Kim, and Daesik Hong. 2021. Deep reinforcement learning-based task offloading decision in the time varying channel. In *Proceedings of the International Conference on Electronics, Information, and Communication (ICEIC'21)*. <https://doi.org/10.1109/iceic51217.2021.9369737>
- [127] Peizhi Yan and Salimur Choudhury. 2021. Deep Q-learning enabled joint optimization of mobile edge computing multi-level task offloading. *Computer Commun.* 180 (2021), 271–283. <https://doi.org/10.1016/j.comcom.2021.09.028>
- [128] Lu Zhang, Zi-Yan Zhang, Luo Min, Chao Tang, Hong-Ying Zhang, Ya-Hong Wang, and Peng Cai. 2021. Task offloading and trajectory control for UAV-assisted mobile edge computing using deep reinforcement learning. *IEEE Access* 9 (2021), 53708–53719. <https://doi.org/10.1109/access.2021.3070908>
- [129] Jianfeng Lv, Jingyu Xiong, Hongzhi Guo, and Jiajia Liu. 2019. Joint computation offloading and resource configuration in ultra-dense edge computing networks: A deep reinforcement learning solution. In *Proceedings of the IEEE 90th Vehicular Technology Conference (VTC2019-Fall)*. <https://doi.org/10.1109/vtcfall.2019.8891384>
- [130] Israr Khan, Xiaofeng Tao, G. M. Shafiqur Rahman, Waheed Ur Rehman, and Tabinda Salam. 2020. Advanced energy-efficient computation offloading using deep reinforcement learning in MTC edge computing. *IEEE Access* 8 (2020), 82867–82875. <https://doi.org/10.1109/access.2020.2991057>
- [131] Tai Manh Ho and Kim-Khoa Nguyen. 2020. Joint server selection, cooperative offloading and handover in multi-access edge computing wireless network: A deep reinforcement learning approach. *IEEE Trans. Mobile Comput.* (2020), 1–1. <https://doi.org/10.1109/tmc.2020.3043736>
- [132] Xianfu Chen, Honggang Zhang, Celimuge Wu, Shiwen Mao, Yusheng Ji, and Mehdi Bennis. 2018. Performance optimization in mobile-edge computing via deep reinforcement learning. In *Proceedings of the IEEE 88th Vehicular Technology Conference (VTC-Fall)*. <https://doi.org/10.1109/vtcfall.2018.8690980>
- [133] Laha Ale, Ning Zhang, Xiaojie Fang, Xianfu Chen, Shaohua Wu, and Longzhuang Li. 2021. Delay-Aware and energy-efficient computation offloading in mobile-edge computing using deep reinforcement learning. *IEEE Trans. Cogn. Commun. Netw.* 7 (2021), 881–892.
- [134] Mohammed Laroui, Hatem Ibn-Khedher, Moussa Ali Cherif, Hassine MOUNGLA, Hossam Afifi, and Ahmed E. Kamel. 2021. SO-VMec: Service offloading in virtual mobile edge computing using deep reinforcement learning. *Trans. Emerg. Telecommun. Technol.* <https://doi.org/10.1002/ett.4211>
- [135] Jie Zhang, Hongzhi Guo, and Jiajia Liu. 2020. Adaptive task offloading in vehicular edge computing networks: A reinforcement learning based scheme. *Mobile Netw. Appl.* 25 (2020), 1736–1745. <https://doi.org/10.1007/s11036-020-01584-6>
- [136] Bing Lin, Kai Lin, Changhang Lin, Yu Lu, Ziqing Huang, and Xinwei Chen. 2021. Computation offloading strategy based on deep reinforcement learning for connected and autonomous vehicle in vehicular edge computing. *J. Cloud Comput.* 10, 1 (2021), 1–17. DOI: <https://doi.org/10.1186/s13677-021-00246-6>
- [137] Jianxi Wang and Liutao Wang. 2021. Mobile edge computing task distribution and offloading algorithm based on deep reinforcement learning in internet of vehicles. *J. Ambient Intell. Human. Comput.* (2021). <https://doi.org/10.1007/s12652-021-03458-5>
- [138] Ziyang Wu and Danfeng Yan. 2021. Deep reinforcement learning-based computation offloading for 5G vehicle-aware multi-access edge computing network. *China Commun.* 18 (2021), 26–41. <https://doi.org/10.23919/jcc.2021.11.003>
- [139] Hado Hasselt, 2010. Double Q-learning. *Adv. Neural Info. Process. Syst.* 23 (2010).
- [140] John Schulman, Philipp Moritz, Sergey Levine, Michael I. Jordan, and Pieter Abbeel. 2015. High-dimensional continuous control using generalized advantage estimation. Retrieved from <https://arXiv:1506.02438>. <https://doi.org/10.48550/arXiv.1506.02438>
- [141] Ziyu Wang, Tom Schaul, Matteo Hessel, Hado Hasselt, Marc Lanctot, and Nando Freitas. 2016. Dueling network architectures for deep reinforcement learning. In *Proceedings of the International Conference on Machine Learning*. <https://doi.org/10.48550/arXiv.1511.06581>
- [142] Matthew Hausknecht and Peter Stone. 2015. Deep recurrent q-learning for partially observable mdps. In *Proceedings of the AAAI Fall Symposium Series*.
- [143] Huan Zhou, Kai Jiang, Xuxun Liu, Xiuhua Li, and Victor C. M. Leung. 2022. Deep reinforcement learning for energy-efficient computation offloading in mobile-edge computing. *IEEE Internet Things J.* 9 (2022), 1517–1530. <https://doi.org/10.1109/jiot.2021.3091142>
- [144] Ming Tang and Vincent W. S. Wong. 2022. Deep reinforcement learning for task offloading in mobile edge computing systems. *IEEE Trans. Mobile Comput.* 21 (2022), 1985–1997. <https://doi.org/10.1109/tmc.2020.3036871>
- [145] Zhaolong Ning, Peiran Dong, Xiaojie Wang, Lei Guo, Joel J. P. C. Rodrigues, Xiangjie Kong, Jun Huang, and Ricky Y. K. Kwok. 2019. Deep reinforcement learning for intelligent internet of vehicles: An energy-efficient

- computational offloading scheme. *IEEE Trans. Cogn. Commun. Netw.* 5 (2019), 1060–1072. <https://doi.org/10.1109/tccn.2019.2930521>
- [146] Y. Wang, K. Wang, H. Huang, T. Miyazaki, and S. Guo. 2018. Traffic and computation co-offloading with reinforcement learning in fog computing for industrial applications. *IEEE Trans. Industr. Inform.* 15, 2 (2018), 976–986.
 - [147] Jinming Shi, Jun Du, Jian Wang, and Jian Yuan. 2020. Distributed V2V computation offloading based on dynamic pricing using deep reinforcement learning. In *Proceedings of the IEEE Wireless Communications and Networking Conference (WCNC'20)*. <https://doi.org/10.1109/wcnc45663.2020.9120816>
 - [148] Zhaolong Ning, Ye Li, Peiran Dong, Xiaojie Wang, Mohammad S. Obaidat, Xiping Hu, Lei Guo, Yi Guo, Jun Huang, and Bin Hu. 2020. When deep reinforcement learning meets 5G-Enabled vehicular networks: A distributed offloading framework for traffic big data. *IEEE Trans. Industr. Inform.* 16 (2020), 1352–1361. <https://doi.org/10.1109/tii.2019.2937079>
 - [149] Daisuke Mochizuki, Yu Abiko, Takato Saito, Daizo Ikeda, and Hiroshi Mineno. 2019. Delay-Tolerance-Based mobile data offloading using deep reinforcement learning. *Sensors* 19 (2019), 1674. <https://doi.org/10.3390/s19071674>
 - [150] Daisuke Mochizuki, Yu Abiko, Hiroshi Mineno, Takato Saito, Daizo Ikeda, and Masaji Katagiri. 2018. Deep reinforcement learning-based method of mobile data offloading. In *Proceedings of the 11th International Conference on Mobile Computing and Ubiquitous Network (ICMU'18)*. <https://doi.org/10.23919/icmu.2018.8653588>
 - [151] Dinh C. Nguyen, Pubudu N. Pathirana, Ming Ding, and Aruna Seneviratne. 2021. Secure computation offloading in blockchain based iot networks with deep reinforcement learning. *IEEE Trans. Netw. Sci. Eng.* 8 (2021), 3192–3208. <https://doi.org/10.1109/tNSE.2021.3106956>
 - [152] Fan Jiang, Rongxin Ma, Youjun Gao, and Zesheng Gu. 2021. A reinforcement learning based computing offloading and resource allocation scheme in F-RAN. <https://doi.org/10.21203/rs.3.rs-483062/v1>.
 - [153] Ronald J. Williams. 1988. On the use of backpropagation in associative reinforcement learning. In *Proceedings of the IEEE International Conference on Neural Networks*. <https://doi.org/10.1109/icnn.1988.23856>.
 - [154] Ronald J. Williams. 1992. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Mach. Learn.* 8, 3 (1992), 229–256.
 - [155] Richard S. Sutton, David McAllester, Satinder Singh, and Yishay Mansour. 1999. Policy gradient methods for reinforcement learning with function approximation. *Adv. Neural Info. Process. Syst.* 12 (1999).
 - [156] Ryan Lowe, Yi Wu, Aviv Tamar, Jean Harb, OpenAI Pieter Abbeel, and Igor Mordatch. 2017. Multi-agent actor-critic for mixed cooperative-competitive environments. *Adv. Neural Info. Process. Syst.* 30 (2017).
 - [157] Vijay Konda and John Tsitsiklis. 1999. Actor-critic algorithms. *Adv. Neural Info. Process. Syst.* 12 (1999).
 - [158] Vijaymohan R. Konda and Vivek S. Borkar. 1999. Actor-critic-type learning algorithms for markov decision processes. *SIAM J. Control Optimiz.* 38 (1999), 94–123. <https://doi.org/10.1137/s036301299731669x>
 - [159] Volodymyr Mnih, Adria Puigdomenech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. 2016. Asynchronous methods for deep reinforcement learning. In *Proceedings of the International Conference on Machine Learning*. 1928–1937.
 - [160] Qi Qi, Jingyu Wang, Zhanyu Ma, Haifeng Sun, Yufei Cao, Lingxin Zhang, and Jianxin Liao. 2019. Knowledge-driven service offloading decision for vehicular edge computing: A deep reinforcement learning approach. *IEEE Trans. Vehic. Technol.* 68 (2019), 4192–4203.
 - [161] Xiaoyu Qiu, Weikun Zhang, Wuhui Chen, and Zibin Zheng. 2021. Distributed and collective deep reinforcement learning for computation offloading: A practical perspective. *IEEE Trans. Parallel Distrib. Syst.* 32 (2021), 1085–1101. <https://doi.org/10.1109/tpds.2020.3042599>
 - [162] Suzhi Bi, Liang Huang, Hui Wang, and Ying-Jun Angela Zhang. 2021. Lyapunov-guided deep reinforcement learning for stable online computation offloading in mobile-edge computing networks. *IEEE Trans. Wireless Commun.* 20, 11 (2021), 7519–7537. DOI: <https://doi.org/10.1109/TWC.2021.3085319>
 - [163] Liang Xiao, Xiaozhen Lu, Tangwei Xu, Xiaoyue Wan, Wen Ji, and Yanyong Zhang. 2020. Reinforcement learning-based mobile offloading for edge computing against jamming and interference. *IEEE Trans. Commun.* 68 (2020), 6114–6126. <https://doi.org/10.1109/tcomm.2020.3007742>
 - [164] Van Dat Tuong, Thanh Phung Truong, The-Vi Nguyen, Wonjong Noh, and Sungrae Cho. 2021. Partial computation offloading in NOMA-Assisted Mobile-Edge computing systems using deep reinforcement learning. *IEEE Internet Things J.* 8 (2021), 13196–13208. <https://doi.org/10.1109/jiot.2021.3064995>
 - [165] Alessio Sacco, Flavio Esposito, Guido Marchetto, and Paolo Montuschi. 2021. Sustainable task offloading in UAV networks via multi-agent reinforcement learning. *IEEE Trans. Vehic. Technol.* 70 (2021), 5003–5015. <https://doi.org/10.1109/tvt.2021.3074304>
 - [166] Peng Lin, Qingyang Song, F. Richard Yu, Dan Wang, and Lei Guo. 2021. Task offloading for wireless VR-enabled medical treatment with blockchain security using collective reinforcement learning. *IEEE Internet Things J.* 8 (2021), 15749–15761. <https://doi.org/10.1109/jiot.2021.3051419>

- [167] Koyela Chakrabarti. 2021. Deep learning-based offloading for mobile augmented reality application in 6G. *Comput. Electr. Eng.* 95 (2021), 107381. <https://doi.org/10.1016/j.compeleceng.2021.107381>
- [168] Yufeng Zhan, Song Guo, Peng Li, and Jiang Zhang. 2020. A deep reinforcement learning based offloading game in edge computing. *IEEE Trans. Comput.* 69, 6 (2020), 883–893. <https://doi.org/10.1109/TC.2020.2969148>
- [169] Yu Chen, Sheng Zhang, Mingjun Xiao, Zhuzhong Qian, Jie Wu, and Sanglu Lu. 2020. Multi-user edge-assisted video analytics task offloading game based on deep reinforcement learning. In *Proceedings of the IEEE 26th International Conference on Parallel and Distributed Systems (ICPADS'20)*.
- [170] John F. Nash. Jr. 1950. Equilibrium points in n-person games. *Proc. Natl. Acad. Sci. U.S.A.* 36, 1 (1950), 48–49.
- [171] Jie Feng, F. Richard Yu, Qingqi Pei, Xiaoli Chu, Jianbo Du, and Li Zhu. 2019. Cooperative computation offloading and resource allocation for blockchain-enabled mobile-edge computing: A deep reinforcement learning approach. *IEEE Internet Things J.* 7, 7 (2019), 6214–6228. <https://doi.org/10.1109/JIOT.2019.2961707>
- [172] Min Hao, Dongdong Ye, Siming Wang, Beihai Tan, and Rong Yu. 2021. URLLC resource slicing and scheduling for trustworthy 6G vehicular services: A federated reinforcement learning approach. *Phys. Commun.* 49 (2021), 101470. <https://doi.org/10.1016/j.phycom.2021.101470>
- [173] John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. 2015. Trust region policy optimization. In *Proceedings of the International Conference on Machine Learning*. 1889–1897.
- [174] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. Retrieved from <https://arXiv:1707.06347>. <https://doi.org/10.48550/arXiv.1707.06347>
- [175] Maxim Lapan. 2020. *Deep Reinforcement Learning Hands-on: Apply Modern RL Methods to PRACTICAL Problems of Chatbots, Robotics, Discrete Optimization, Web Automation, and More*. Packt Publishing Ltd.
- [176] Andrew Ilyas, Logan Engstrom, Shibani Santurkar, Dimitris Tsipras, Firdaus Janoo, Larry Rudolph, and Aleksander Madry. 2018. A closer look at deep policy gradients. Retrieved from <https://arXiv:1811.02553>. <https://doi.org/10.48550/arXiv.1811.02553>
- [177] Jin Wang, Jia Hu, Geyong Min, Wenhan Zhan, Albert Zomaya, and Nektarios Georgalas. 2021. Dependent task offloading for edge computing based on deep reinforcement learning. *IEEE Trans. Comput.* (2021). <https://doi.org/10.1109/TC.2021.3131040>
- [178] Shuyang Li, Xiaohui Hu, and Yongwen Du. 2021. Deep reinforcement learning and game theory for computation offloading in dynamic edge computing markets. *IEEE Access* 9 (2021), 121456–121466. <https://doi.org/10.1109/access.2021.3109132>
- [179] Wenhan Zhan, Chunbo Luo, Jin Wang, Geyong Min, and Hancong Duan. 2020. Deep reinforcement learning-based offloading scheduling for vehicular edge computing. *IEEE Internet Things J.* (2020). <https://doi.org/10.1109/JIOT.2020.2978830>
- [180] Timothy P. Lillicrap, Jonathan J. Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. 2015. Continuous control with deep reinforcement learning. Retrieved from <https://arXiv:1509.02971>. <https://doi.org/10.48550/arXiv.1509.02971>
- [181] David Silver, Guy Lever, Nicolas Heess, Thomas Degris, Daan Wierstra, and Martin Riedmiller. 2014. Deterministic policy gradient algorithms. In *Proceedings of the International Conference on Machine Learning*. 387–395.
- [182] Yunzhao Li, Feng Qi, Zhili Wang, Xiuming Yu, and Sujie Shao. 2020. Distributed edge computing offloading algorithm based on deep reinforcement learning. *IEEE Access* 8 (2020), 85204–85215. <https://doi.org/10.1109/access.2020.2991773>
- [183] Haodong Lu, Xiaoming He, Miao Du, Xiukai Ruan, Yanfei Sun, and Kun Wang. 2020. Edge QoE: Computation offloading with deep reinforcement learning for internet of things. *IEEE Internet Things J.* 7 (2020), 9255–9265. <https://doi.org/10.1109/jiot.2020.2981557>
- [184] Juan Chen and Zongling Wu. 2021. Dynamic computation offloading with energy harvesting devices: A graph-based deep reinforcement learning approach. *IEEE Commun. Lett.* 25 (2021), 2968–2972. <https://doi.org/10.1109/lcomm.2021.3094842>
- [185] Xiaoxu Zhong and Yejun He. 2021. A cybertwin-driven task offloading scheme based on deep reinforcement learning and graph attention networks. In *Proceedings of the 13th International Conference on Wireless Communications and Signal Processing (WCSP'21)*. <https://doi.org/10.1109/wcsp52459.2021.9613687>
- [186] Yao Zhu, Yulin Hu, Tianyu Yang, Tao Yang, Jannik Vogt, and Anke Schmeink. 2021. Reliability-optimal offloading in low-latency edge computing networks: Analytical and reinforcement learning based designs. *IEEE Trans. Vehic. Technol.* 70 (2021), 6058–6072. <https://doi.org/10.1109/tvt.2021.3073791>
- [187] Abegaz Mohammed Seid, Gordon Owusu Boateng, Stephen Anokye, Thomas Kwantwi, Guolin Sun, and Guisong Liu. 2021. Collaborative computation offloading and resource allocation in Multi-UAV-Assisted IoT networks: A deep reinforcement learning approach. *IEEE Internet Things J.* 8 (2021), 12203–12218. <https://doi.org/10.1109/jiot.2021.3063188>

- [188] Ang Gao, Qi Wang, Kaiyue Chen, and Wei Liang. 2021. Multi-UAV assisted offloading optimization: A game combined reinforcement learning approach. *IEEE Commun. Lett.* 25 (2021), 2629–2633. <https://doi.org/10.1109/lcomm.2021.3078469>
- [189] Yunpeng Wang, Weiwei Fang, Yi Ding, and Naixue Xiong. 2021. Computation offloading optimization for UAV-assisted mobile edge computing: A deep deterministic policy gradient approach. *Wireless Netw.* 27 (2021), 2991–3006. <https://doi.org/10.1007/s11276-021-02632-z>
- [190] Zilong Cao, Pan Zhou, Ruixuan Li, Siqi Huang, and Dapeng Wu. 2020. Multiagent deep reinforcement learning for joint multichannel access and task offloading of mobile-edge computing in industry 4.0. *IEEE Internet Things J.* 7 (2020), 6201–6213. <https://doi.org/10.1109/jiot.2020.2968951>
- [191] Jakob Foerster, Ioannis Alexandros Assael, Nando de Freitas, and Shimon Whiteson. 2016. *Adv. Neural Info. Process. Syst.* 29 (2016).
- [192] Xiaoyan Huang, Supeng Leng, Sabita Maharjan, and Yan Zhang. 2021. Multi-Agent deep reinforcement learning for computation offloading and interference coordination in small cell networks. *IEEE Trans. Vehic. Technol.* 70 (2021), 9282–9293. <https://doi.org/10.1109/tvt.2021.3096928>
- [193] Zhao Chen, Lei Zhang, Yukui Pei, Chunxiao Jiang, and Liuguo Yin. 2022. NOMA-Based Multi-User mobile edge computation offloading via cooperative Multi-Agent deep reinforcement learning. *IEEE Trans. Cogn. Commun. Netw.* 8 (2022), 350–364.
- [194] Xing Chen and Guizhong Liu. 2020. Joint optimization of task offloading and resource allocation via deep reinforcement learning for augmented reality in mobile edge network. In *Proceedings of the IEEE International Conference on Edge Computing (EDGE'20)*. <https://doi.org/10.1109/edge50951.2020.00019>
- [195] Wenjing Hou, Hong Wen, Huanhuan Song, Wenxin Lei, and Wei Zhang. 2021. Multiagent deep reinforcement learning for task offloading and resource allocation in cybertwin-based networks. *IEEE Internet Things J.* 8 (2021), 16256–16268. <https://doi.org/10.1109/jiot.2021.3095677>
- [196] Xinyu Huang, Lijun He, and Wanyue Zhang. 2020. Vehicle speed aware computing task offloading and resource allocation based on multi-agent reinforcement learning in a vehicular edge computing network. In *Proceedings of the IEEE International Conference on Edge Computing (EDGE'20)*.
- [197] Tuomas Haarnoja, Aurick Zhou, Kristian Hartikainen, George Tucker, Sehoon Ha, Jie Tan, Vikash Kumar, Henry Zhu, Abhishek Gupta, Pieter Abbeel, and Sergey Levine. 2018. Soft actor-critic algorithms and applications. Retrieved from <https://arXiv:1812.05905>. <https://doi.org/10.48550/arXiv.1812.05905>
- [198] Jinming Shi, Jun Du, Jingjing Wang, Jian Wang, and Jian Yuan. 2020. Priority-aware task offloading in vehicular fog computing based on deep reinforcement learning. *IEEE Trans. Vehic. Technol.* 69 (2020), 16067–16081. <https://doi.org/10.1109/tvt.2020.3041929>
- [199] Jinming Shi, Jun Du, Jian Wang, and Jian Yuan. 2021. Deep reinforcement learning-based V2V partial computation offloading in vehicular fog computing. In *Proceedings of the IEEE Wireless Communications and Networking Conference (WCNC'21)*. <https://doi.org/10.1109/wcnc49053.2021.9417450>
- [200] Dawei Chen, Yin-Chen Liu, BaekGyu Kim, Jiang Xie, Choong Seon Hong, and Zhu Han. 2020. Edge computing resources reservation in vehicular networks: A meta-learning approach. *IEEE Trans. Vehic. Technol.* 69 (2020), 5634–5646. <https://doi.org/10.1109/tvt.2020.2983445>
- [201] Marcin Andrychowicz, Misha Denil, Sergio Gómez, Matthew W. Hoffman, David Pfau, Tom Schaul, Brendan Shillingford, and Nando de Freitas. 2016. Learning to learn by gradient descent by gradient descent. *Adv. Neural Info. Process. Syst.* 29 (2016).
- [202] Abhishek Gupta, Russell Mendonca, YuXuan Liu, Pieter Abbeel, and Sergey Levine. 2018. Meta-reinforcement learning of structured exploration strategies. *Adv. Neural Info. Process. Syst.* 31 (2018).
- [203] Chelsea Finn, Pieter Abbeel, and Sergey Levine. 2017. Model-agnostic meta-learning for fast adaptation of deep networks. In *Proceedings of the International Conference on Machine Learning*. 1126–1135.
- [204] J. X. Wang, Z. Kurth-Nelson, D. Tirumala, H. Soyer, J. Z. Leibo, R. Munos, C. Blundell, D. Kumaran, and M. Botvinick. 2016. Learning to reinforcement learn. Retrieved from <https://arXiv:1611.05763>. <https://doi.org/10.48550/arXiv.1611.05763>
- [205] Guanjin Qu, Huaming Wu, Ruidong Li, and Pengfei Jiao. 2021. DMRO: A deep meta reinforcement learning-based task offloading framework for Edge-Cloud computing. *IEEE Trans. Netw. Serv. Manag.* 18 (2021), 3448–3459. <https://doi.org/10.1109/tnsm.2021.3087258>
- [206] Jin Wang, Jia Hu, Geyong Min, Albert Y. Zomaya, and Nektarios Georgalas. 2021. Fast adaptive task offloading in edge computing based on meta reinforcement learning. *IEEE Trans. Parallel Distrib. Syst.* 32 (2021), 242–253. <https://doi.org/10.1109/tpds.2020.3014896>
- [207] Alia Asheralieva and Dusit Niyato. 2019. Hierarchical game-theoretic and reinforcement learning framework for computational offloading in UAV-Enabled mobile edge computing networks with multiple service providers. *IEEE Internet Things J.* 6 (2019), 8753–8769. <https://doi.org/10.1109/jiot.2019.2923702>

- [208] Ziling Wei, Baokang Zhao, Jinshu Su, and Xicheng Lu. 2019. Dynamic edge computation offloading for internet of things with energy harvesting: A learning method. *IEEE Internet Things J.* 6 (2019), 4436–4447. <https://doi.org/10.1109/jiot.2018.2882783>
- [209] Liang Huang, Suzhi Bi, and Ying-Jun Angela Zhang. 2019. Deep reinforcement learning for online computation offloading in wireless powered mobile-edge computing networks. *IEEE Trans. Mobile Comput.* 19, 11 (2019), 2581–2593. <https://doi.org/10.1109/TMC.2019.2928811>
- [210] Maurice Nduwayezu, Quoc-Viet Pham, and Won-Joo Hwang. 2020. Online computation offloading in NOMA-Based Multi-Access edge computing: A deep reinforcement learning approach. *IEEE Access* 8 (2020), 99098–99109. <https://doi.org/10.1109/access.2020.2997925>
- [211] Xiucheng Wang and Hongzhi Guo. 2021. Mobility-Aware computation offloading for swarm robotics using deep reinforcement learning. In *Proceedings of the IEEE 18th Annual Consumer Communications and Networking Conference (CCNC'21)*. <https://doi.org/10.1109/ccnc49032.2021.9369456>
- [212] Feifei Zhang, Jidong Ge, Chifong Wong, Chuanyi Li, Xingguo Chen, Sheng Zhang, Bin Luo, He Zhang, and Victor Chang. 2019. Online learning offloading framework for heterogeneous mobile edge computing system. *J. Parallel Distrib. Comput.* 128 (2019), 167–183.
- [213] Yi Liu, Huimin Yu, Shengli Xie, and Yan Zhang. 2019. Deep reinforcement learning for offloading and resource allocation in vehicle edge computing and networks. *IEEE Trans. Vehic. Technol.* 68, 11 (2019), 11158–11168. <https://doi.org/10.1109/TVT.2019.2935450>
- [214] Hao Hao, Changqiao Xu, Lujie Zhong, and Gabriel-Miro Muntean. 2020. A multi-update deep reinforcement learning algorithm for edge computing service offloading. In *Proceedings of the 28th ACM International Conference on Multimedia*. <https://doi.org/10.1145/3394171.3413702>
- [215] Li Quan, Zhiliang Wang, and Fuji Ren. 2018. A novel Two-Layered reinforcement learning for task offloading with tradeoff between physical machine utilization rate and delay. *Future Internet* 10 (2018), 60. <https://doi.org/10.3390/fi10070060>
- [216] Fatemeh Jazayeri, Ali Shahidinejad, and Mostafa Ghobaei-Arani. 2020. Autonomous computation offloading and auto-scaling the in the mobile fog computing: A deep reinforcement learning-based approach. *J. Ambient Intell. Human. Comput.* 12 (2020), 8265–8284.
- [217] Ismail AlQerm and Basem Shihada. 2019. Energy efficient traffic offloading in multi-tier heterogeneous 5G Networks using intuitive online reinforcement learning. *IEEE Trans. Green Commun. Network.* 3 (2019), 691–702. <https://doi.org/10.1109/tgcn.2019.2916900>
- [218] Xiaoyu Qiu, Luobin Liu, Wuhui Chen, Zicong Hong, and Zibin Zheng. 2019. Online deep reinforcement learning for computation offloading in blockchain-empowered mobile edge computing. *IEEE Trans. Vehic. Technol.* 68 (2019), 8050–8062. <https://doi.org/10.1109/tvt.2019.2924015>
- [219] Minghui Min, Liang Xiao, Ye Chen, Peng Cheng, Di Wu, and Weihua Zhuang. 2019. Learning-based computation offloading for IoT devices with energy harvesting. *IEEE Trans. Vehic. Technol.* 68, 2 (2019), 1930–1941. <https://doi.org/10.1109/TVT.2018.2890685>
- [220] Sukhpal Singh Gill, Minxian Xu, Carlo Ottaviani, Panos Patros, Rami Bahsoon, Arash Shaghaghi, Muhammed Golec, et al. 2022. AI for next generation computing: Emerging trends and future directions. *Internet Things* 19 (2022), 100514. <https://doi.org/10.1016/j.iot.2022.100514>
- [221] Baldini Ioana, Paul Castro, Kerry Chang, Perry Cheng, Stephen Fink, Vatche Ishakian, Nick Mitchell, et al. 2017. Serverless computing: Current trends and open problems. In *Research Advances in Cloud Computing*. Springer, Singapore, 1–20. https://doi.org/10.1007/978-981-10-5026-8_1
- [222] Ujjawal Maan and Yogesh Chaba. 2021. Deep Q-network based fog node offloading strategy for 5 G vehicular Adhoc Network. *Ad Hoc Netw.* 120, 102565.
- [223] Huan Zhou, Tong Wu, Haijun Zhang, and Jie Wu. 2021. Incentive-Driven deep reinforcement learning for content caching and D2D offloading. *IEEE J. Select. Areas Commun.* 39 (2021), 2445–2460. <https://doi.org/10.1109/jsac.2021.3087232>
- [224] Bo Gu, Yifei Wei, Xiaowei Liu, Mei Song, and Zhu Han. 2019. Traffic offloading and power allocation for green hetnets using reinforcement learning method. In *Proceedings of the IEEE Global Communications Conference (GLOBECOM'19)*. <https://doi.org/10.1109/globecom38437.2019.9014245>
- [225] Qihe Huang, Xiaolong Xu, and Jinhui Chen. 2021. Learning-aided fine-grained offloading for real-time applications in edge-cloud computing. *Wireless Netw.* (2021). <https://doi.org/10.1007/s11276-021-02750-8>
- [226] Tao Leng, Pengfei Duan, Dongwei Hu, Gaofeng Cui, and Weidong Wang. 2021. Cooperative user association and resource allocation for task offloading in hybrid GEO-LEO satellite networks. *Int. J. Satellite Commun. Netw.* 40 (2021), 230–243. <https://doi.org/10.1002/sat.1436>
- [227] Hao Tian, Xiaolong Xu, Lianying Qi, Xuyun Zhang, Wanchun Dou, Shui Yu, and Qiang Ni. 2021. CoPace: Edge computation offloading and caching for self-driving with deep reinforcement learning. *IEEE Trans. Vehic. Technol.* 70 (2021), 13281–13293. <https://doi.org/10.1109/tvt.2021.3121096>

- [228] Dawei Wei, Jianfeng Ma, Linbo Luo, Yunbo Wang, Lei He, and Xinghua Li. 2021. Computation offloading over multi-UAV MEC network: A distributed deep reinforcement learning approach. *Comput. Netw.* 199 (2021), 108439. <https://doi.org/10.1016/j.comnet.2021.108439>
- [229] Xiaoming He, Haodong Lu, Miao Du, Yingchi Mao, and Kun Wang. 2021. QoE-Based task offloading with deep reinforcement learning in edge-enabled internet of vehicles. *IEEE Trans. Intell. Transport. Syst.* 22 (2021), 2252–2261. <https://doi.org/10.1109/tits.2020.3016002>
- [230] Xiaoming He, Haodong Lu, Yingchi Mao, and Kun Wang. 2020. QoE-driven task offloading with deep reinforcement learning in edge intelligent IoT. In *Proceedings of the IEEE Global Communications Conference (GLOBECOM'20)*. <https://doi.org/10.1109/globecom42002.2020.9348050>
- [231] Xiaoming He, Haodong Lu, Huawei Huang, Yingchi Mao, Kun Wang, and Song Guo. 2020. QoE-Based cooperative task offloading with deep reinforcement learning in mobile edge networks. *IEEE Wireless Commun.* 27 (2020), 111–117. <https://doi.org/10.1109/mwc.001.1900406>
- [232] Ang Gao, Qi Wang, Wei Liang, and Zhiguo Ding. 2021. Game combined multi-agent reinforcement learning approach for UAV assisted offloading. *IEEE Trans. Vehic. Technol.* 70 (2021), 12888–12901. <https://doi.org/10.1109/tvt.2021.3121281>
- [233] OpenAI. 2019. Welcome to Spinning Up in Deep RL–Spinning Up documentation. Retrieved from <https://spinningup.openai.com/en/latest/index.html>. Last Accessed: June 20.
- [234] Ashkan Yousefpour, Ishigaki Genya, Gour Riti, and P. Jue Jason. 2018. On reducing IoT service delay via fog offloading. *IEEE Internet Things J.* 5, 2 (2018), 998–1010. DOI: [10.1109/JIOT.2017.2788802](https://doi.org/10.1109/JIOT.2017.2788802)
- [235] Yaru Fu, Xiaolong Yang, Peng Yang, Angus K. Y. Wong, Zheng Shi, Hong Wang, and Tony Q. S. Quek. 2021. Energy-efficient offloading and resource allocation for mobile edge computing enabled mission-critical internet-of-things systems. *EURASIP J. Wireless Commun. Netw.* (2021), 1–16.
- [236] Sukhpal Singh Gill, Peter Garraghan, and Rajkumar Buyya. 2019. ROUTER: Fog enabled cloud based intelligent resource management approach for smart home IoT devices. *J. Syst. Softw.* 154 (2019), 125–138. DOI: <https://doi.org/10.1016/j.jss.2019.04.058>
- [237] Harshit Gupta, Amir Vahid Dastjerdi, Soumya K. Ghosh, and Rajkumar Buyya. 2017. iFogSim: A toolkit for modeling and simulation of resource management techniques in the Internet of Things, Edge and Fog computing environments. *Softw.: Pract. Exper.* 47, 9 (2017), 1275–1296.
- [238] Chen Chen, Haoifei Li, Huan Li, Rufeif Fu, Yangyang Liu, and Shaohua Wan. 2022. Efficiency and fairness oriented dynamic task offloading in internet of vehicles. *IEEE Trans. Green Commun. Network.* 6, 3 (2022), 1481–1493. DOI: [10.1109/TGCN.2022.3167643](https://doi.org/10.1109/TGCN.2022.3167643)
- [239] Atta ur Rehman Khan, Mazliza Othman, Sajjad Ahmad Madani, and Samee Ullah Khan. 2013. A survey of mobile cloud computing application models. *IEEE Commun. Surveys Tutor.* 16, 1 (2013), 393–413. DOI: [10.1109/SURV.2013.062613.00160](https://doi.org/10.1109/SURV.2013.062613.00160)
- [240] Ju Ren, Deyu Zhang, Shiwen He, Yaoxue Zhang, and Tao Li. 2019. A survey on end-edge-cloud orchestrated network computing paradigms: Transparent computing, mobile edge computing, fog computing, and cloudlet. *ACM Comput. Surveys* 52, 6 (2019), 1–36. DOI: <https://doi.org/10.1145/3362031>
- [241] Sunday Oyinlola Ogundoyin and Ismaila Adeniyi Kamil. 2021. Optimization techniques and applications in fog computing: An exhaustive survey. *Swarm Evolution. Comput.* 66 (2021), 100937.
- [242] Huaming Wu. 2018. Multi-objective decision-making for mobile cloud offloading: A survey. *IEEE Access* 6 (2018), 3962–3976. DOI: [10.1109/ACCESS.2018.2791504](https://doi.org/10.1109/ACCESS.2018.2791504)
- [243] Nidhi Kumari, Anirudh Yadav, and Prasanta K. Jana. 2022. Task offloading in fog computing: A survey of algorithms and optimization techniques. *Comput. Netw.* 214 (2022), 109137. <https://doi.org/10.1016/j.comnet.2022.109137>
- [244] R. S. Michalski, J. G. Carbonell, and T. M. Mitchell. (Eds.). 2013. Machine learning: An artificial intelligence approach. *Springer Science & Business Media*. DOI: <https://doi.org/10.1007/978-3-662-12405-5>
- [245] Liu Li, Yuanyuan Du, Qi Fan, and Weicun Zhang. 2019. A survey on computation offloading in the mobile cloud computing environment. *Int. J. Comput. Appl. Technol.* 59, 2 (2019), 106–113. DOI: <https://doi.org/10.1504/IJCAT.2019.098031>
- [246] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu. 2016. Edge computing: Vision and challenges. *IEEE Internet Things J.* 3, 5 (2016), 637–646.
- [247] W. Shi and S. Dustdar. 2016. The promise of edge computing. *Computer* 49, 5 (2016), 78–81. DOI: [10.1109/MC.2016.145](https://doi.org/10.1109/MC.2016.145)
- [248] Ashkan Yousefpour, Caleb Fung, Tam Nguyen, Krishna Kadiyala, Fatemeh Jalali, Amirreza Niakanlahiji, Jian Kong, and Jason P. Jue. 2019. All one needs to know about fog computing and related edge computing paradigms: A complete survey. *J. Syst. Architect.* 98 (2019), 289–330.
- [249] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. Van Den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, and S. Dieleman. 2016. Mastering the game of Go with deep neural networks and tree search. *Nature* 529, 7587 (2016), 484–489. DOI: <https://doi.org/10.1038/nature16961>

- [250] D. Silver, T. Hubert, J. Schrittwieser, I. Antonoglou, M. Lai, A. Guez, M. Lanctot, L. Sifre, D. Kumaran, T. Graepel, and T. Lillicrap. 2018. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science* 362, 6419 (2018), 1140–1144. DOI : [10.1126/science.aar6404](https://doi.org/10.1126/science.aar6404)
- [251] Hamed Shah-Mansouri and Vincent W. S. Wong. 2018. Hierarchical fog-cloud computing for IoT systems: A computation offloading game. *IEEE Internet Things J.* 5, 4 (2018), 3246–3257. DOI : [10.1109/JIOT.2018.2838022](https://doi.org/10.1109/JIOT.2018.2838022)
- [252] Susheel Kumar Joshi and Jagdish Chand Bansal. 2020. Parameter tuning for meta-heuristics. *Knowl.-Based Syst.* 189 (2020), 105094. DOI : <https://doi.org/10.1016/j.knosys.2019.105094>

Received 2 July 2022; revised 29 March 2023; accepted 31 May 2023